Concert Ticketing - Low Level Design

Thiết kế chi tiết dữ liệu và luồng tính năng

Thông tin	Nội dung
Phiên bản	1.0 - 03/11/2026; thiết kế đề xuất để triển khai, chưa phải schema đã chạy.
Phạm vi	5 microservices; local Docker Compose; Java 25 / Spring Boot theo HLD.
Nguồn	BRD, Process Flows, SRS và HLD Concert Ticketing v1.0 ngày 03/11/2026.
Mẫu trình bày	EIGP-EIG Low level design.pdf: document control, baseline, ERD, state model, từng flow, invariant và error model. Chỉ tham khảo cấu trúc; không dùng nghiệp vụ EIG.
Người đọc	Backend developer, frontend developer, QA và SA.
Ngoài phạm vi	Code triển khai, hạ tầng cloud/K8s, PDF vé, chuyển nhượng, hủy concert, hoàn từng vé và check-in offline.

1. Mục tiêu và nguyên tắc

Tài liệu cụ thể hóa HLD thành bảng dữ liệu, khóa/quan hệ, transaction và hợp đồng xử lý theo tính năng. Không thêm service. Các lựa chọn vật lý mới trong LLD là quyết định thiết kế của bản này, không được mô tả là tính năng đã triển khai hoặc đã nghiệm thu.

Khi các nguồn khác nhau, dùng các điều chỉnh được ghi rõ tại HLD mục 1.1 và 18: VNPAY sandbox; Keycloak cho mọi vai trò; đăng nhập email hoặc số điện thoại + mật khẩu; SMS OTP thật; email sandbox kèm QR; không PDF vé; không lọc giá; admin/organizer không xem QR. Những yêu cầu khác của SRS vẫn giữ nguyên.

Mục	Nội dung
2-3	Baseline, quy ước dữ liệu và ERD
4-8	Từ điển dữ liệu Booking, Payment, Notification, Search, Report
9-10	Bảng kỹ thuật, trạng thái và quy tắc cạnh tranh
11	Thiết kế từng nhóm tính năng F01-F12
12-14	API/event, migration, phục hồi và truy vết

2. Baseline kiến trúc

Thành phần	Kiến trúc / cấu trúc	Dữ liệu sở hữu
Booking	Hexagonal; catalog, inventory, ordering, ticketing. Phân quyền/hỗ trợ là application capability trong Booking.	booking_db / PostgreSQL
Payment	Hexagonal; payment và refund. VNPAY là outbound adapter.	payment_db / PostgreSQL
Notification	Hexagonal gọn; email use case + adapter.	notification_db / PostgreSQL
Search	Layered: controller, service, repository/consumer.	Elasticsearch: concert index
Report	Layered API; ingestion/exporter; PySpark batch riêng.	MongoDB + raw objects MinIO
Identity / orchestration	Keycloak + SMS extension; Temporal worker trong Booking.	DB nội bộ do sản phẩm quản lý; không FK đến DB nghiệp vụ.

Trong mỗi module Hexagonal: domain/model và domain/port; application/usecase và application/port; infrastructure/persistence, rest, messaging, external. Domain không import Spring/JPA/Kafka/Temporal. Application quyết định transaction; adapter triển khai port. Không thêm generic base repository hoặc framework workflow riêng.

2.1. Quy ước vật lý

Quy ước	Thiết kế
Tên / định danh	snake_case. PK uuid do ứng dụng sinh; subject Keycloak varchar(128), không giả định subject luôn có dạng UUID. Không dùng email làm PK.
Null / mặc định	Cột Null=N là NOT NULL; Y cho phép null. Không có default trừ khi ghi rõ. Các cột liệt kê cùng nhóm kiểu phải được tạo thành từng cột riêng.
Cột chung M	created_at timestamptz NOT NULL DEFAULT clock_timestamp(); updated_at cùng kiểu/default; version bigint NOT NULL DEFAULT 0 CHECK version >= 0. Mỗi thay đổi có version+1, updated_at theo DB.
Cột chung A	created_at timestamptz NOT NULL DEFAULT clock_timestamp(). Bản ghi append-only; không version nếu không ghi rõ.
Tiền / thời gian	bigint VND nguyên đồng, >0 cho giá/thu/hoàn; phí=0. Không float. UTC lưu trữ/API ISO-8601 có offset; business date Asia/Ho_Chi_Minh.
PK / UK / FK	PK là khóa chính; UK là unique. Tất cả FK vật lý chỉ trong một database, ON DELETE RESTRICT; index cột FK nếu chưa được phủ bởi PK/UK. Không cascade xóa giao dịch.
Enum	varchar + CHECK với tập trạng thái trong mục 10. Không dùng PostgreSQL enum để tránh khó mở rộng migration.
Concurrency	READ COMMITTED + row lock có thứ tự + điều kiện update. Không dùng Redis lock làm căn cứ bán ghế. version phục vụ optimistic write/config và projection.
Deletion	Không tự xóa đơn, tiền, vé, audit, idempotency trong MVP. Ngừng cấu hình ghế bằng is_enabled=false, giữ row cho lịch sử. Reset local là tác vụ vận hành riêng.

2.2. Bất biến và điểm cưỡng chế

Bất biến	Cưỡng chế
Một ghế chỉ bán một lần	Một row seat_inventory/seat; SOLD không có transition ngược; ticket UK(performance_seat_id); xác nhận local transaction.
1-4 ghế, cùng suất, đủ toàn bộ	CHECK seat_count 1..4; composite FK; kiểm đếm lines/tổng tiền trong use case cùng transaction. CHECK SQL đơn lẻ không kiểm tổng liên bảng.
Không thu/hoàn lại vì retry	Khóa payment_order; một attempt đang xử lý/chưa rõ hoặc thành công; reference provider ổn định; UK refund(payment_attempt_id).
Một lần vào cổng	UPDATE ticket có điều kiện ISSUED; check_in PK(ticket_id); hai thao tác trong một transaction.
Không mất side effect sau commit	Outbox/intent + inbox; lưu kết quả nghiệp vụ cùng transaction; dispatcher retry với business key.
Không lộ QR	Token mã hóa + digest; endpoint owner riêng; DTO hỗ trợ không có token; event/audit không có QR.

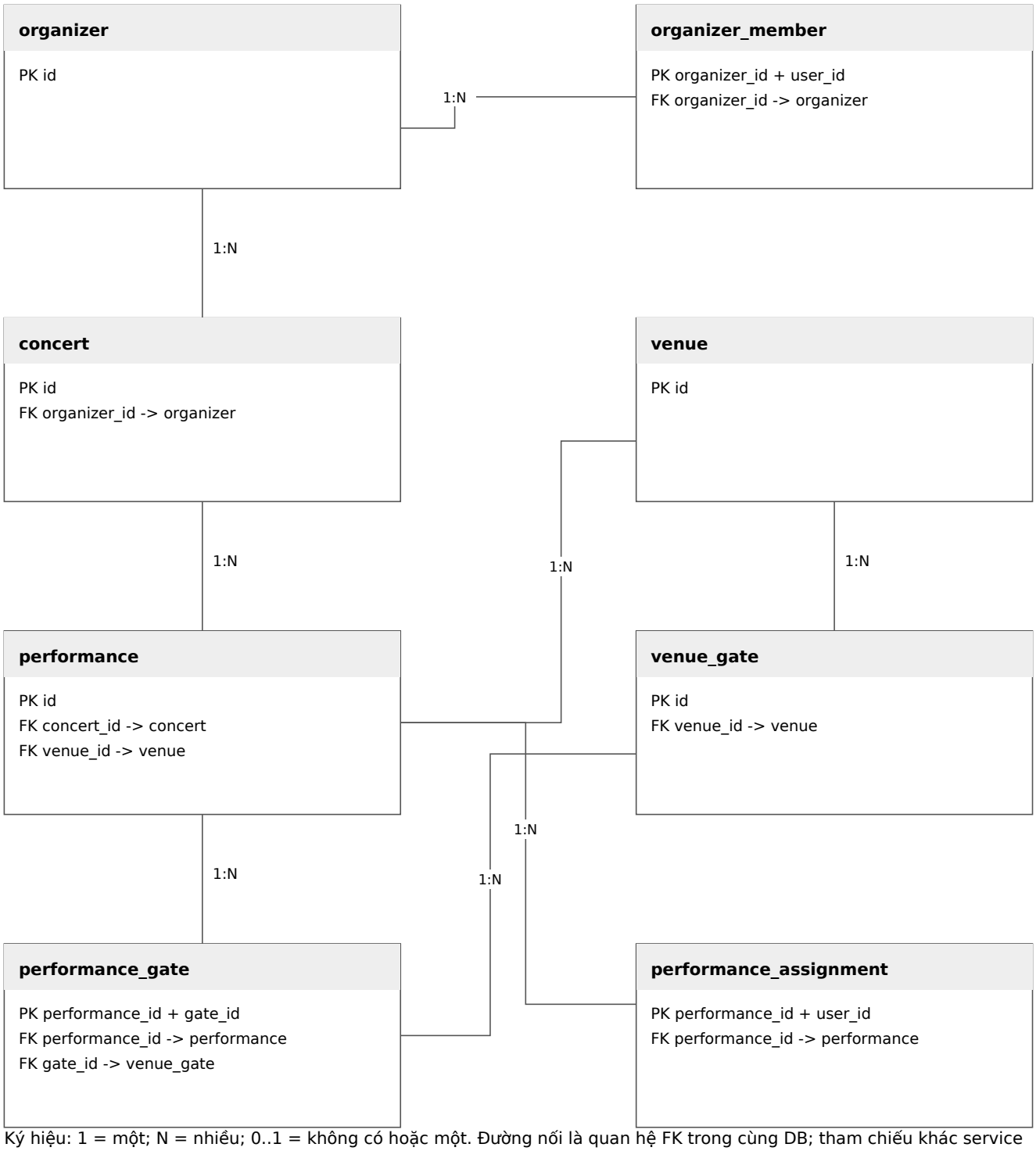
3. Mô hình quan hệ

Mỗi sơ đồ là một lát cắt của cùng mô hình. Một số bảng lặp lại ở sơ đồ kế tiếp để đọc được quan hệ; không tạo hai bản bảng. Không có bảng hold độc lập: booking_order lưu hold_id/held_at/expires_at; seat_inventory lưu quyền giữ hiện tại. order_item giữ lịch sử các ghế của mọi đơn, kể cả đơn hết hạn.

Booking có 4 schema: catalog, inventory, ordering, ticketing; các bảng quyền/hỗ trợ/kỹ thuật đặt trong schema platform. Tên ngắn trong sơ đồ và từ điển được ánh xạ theo chủ sở hữu trong từng mục. Payment và Notification dùng schema public trong DB riêng. Các bảng chung ở mục 9 được tạo riêng trong DB cần dùng.

3.1. ERD Catalog và phân công

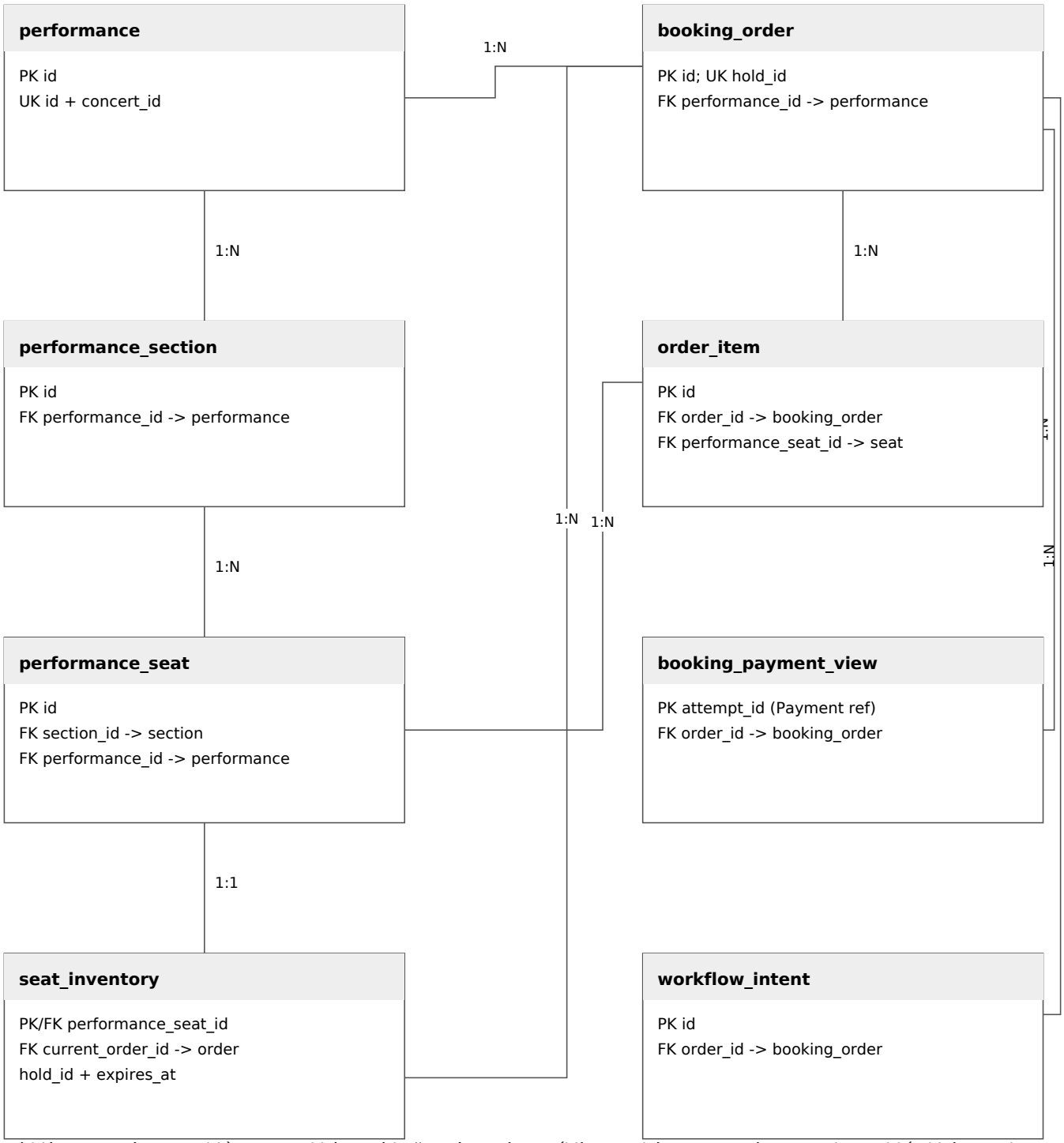
user_id là subject ngoài DB; Booking không chứa password. Composite FK trong performance_gate bảo đảm cổng thuộc đúng venue của performance.



Ký hiệu: 1 = một; N = nhiều; 0..1 = không có hoặc một. Đường nối là quan hệ FK trong cùng DB; tham chiếu khác service chỉ được ghi bằng ref, không có FK vật lý. Sơ đồ chỉ hiện khóa chính/khóa liên kết; từ điển dữ liệu bên dưới quy định đầy đủ cột.

3.2. ERD Inventory và Ordering

Giá đơn lưu bất biến trong order_item; seat_inventory chỉ lưu quyền hiện tại. Mỗi ghế có đúng một row inventory. Các quan hệ ghép theo performance được quy định trong từ điển dữ liệu.

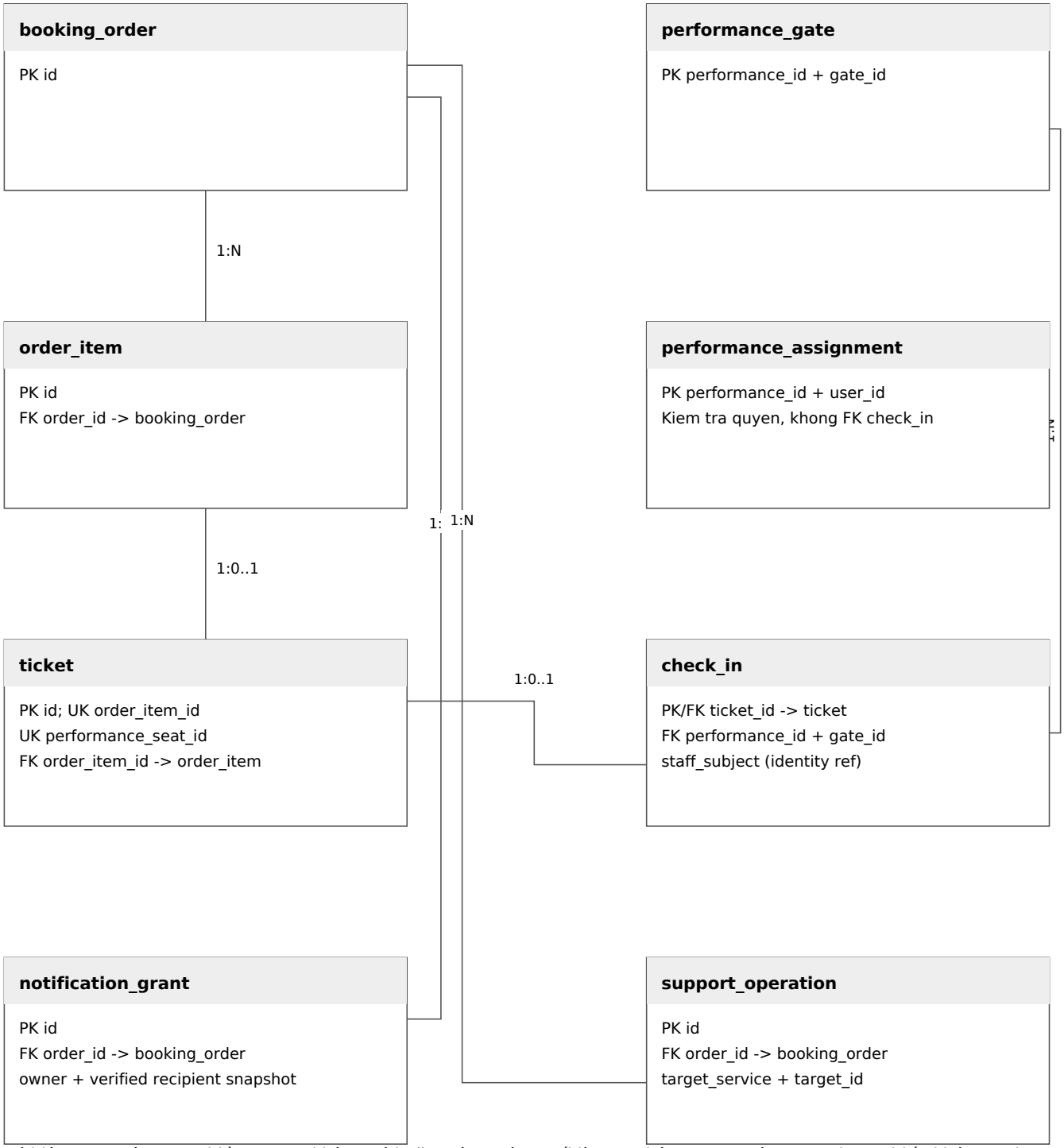


Ký hiệu: 1 = một; N = nhiều; 0..1 = không có hoặc một. Đường nối là quan hệ FK trong cùng DB; tham chiếu khác service chỉ được ghi bằng ref, không có FK vật lý. Sơ đồ chỉ hiện khóa chính/khóa liên kết; từ điển dữ liệu bên dưới quy định đầy đủ cột.

seat_inventory.current_order_id tham chiếu booking_order: một đơn có 0-4 quyền ghế hiện tại, một ghế có 0..1 đơn hiện tại. order_item là lịch sử nên một ghế có thể xuất hiện ở nhiều đơn đã hủy/hết hạn. booking_payment_view và workflow_intent tham chiếu đơn bằng FK; current_order_id/hold_id chỉ được đổi trong Inventory use case.

3.3. ERD Ticketing

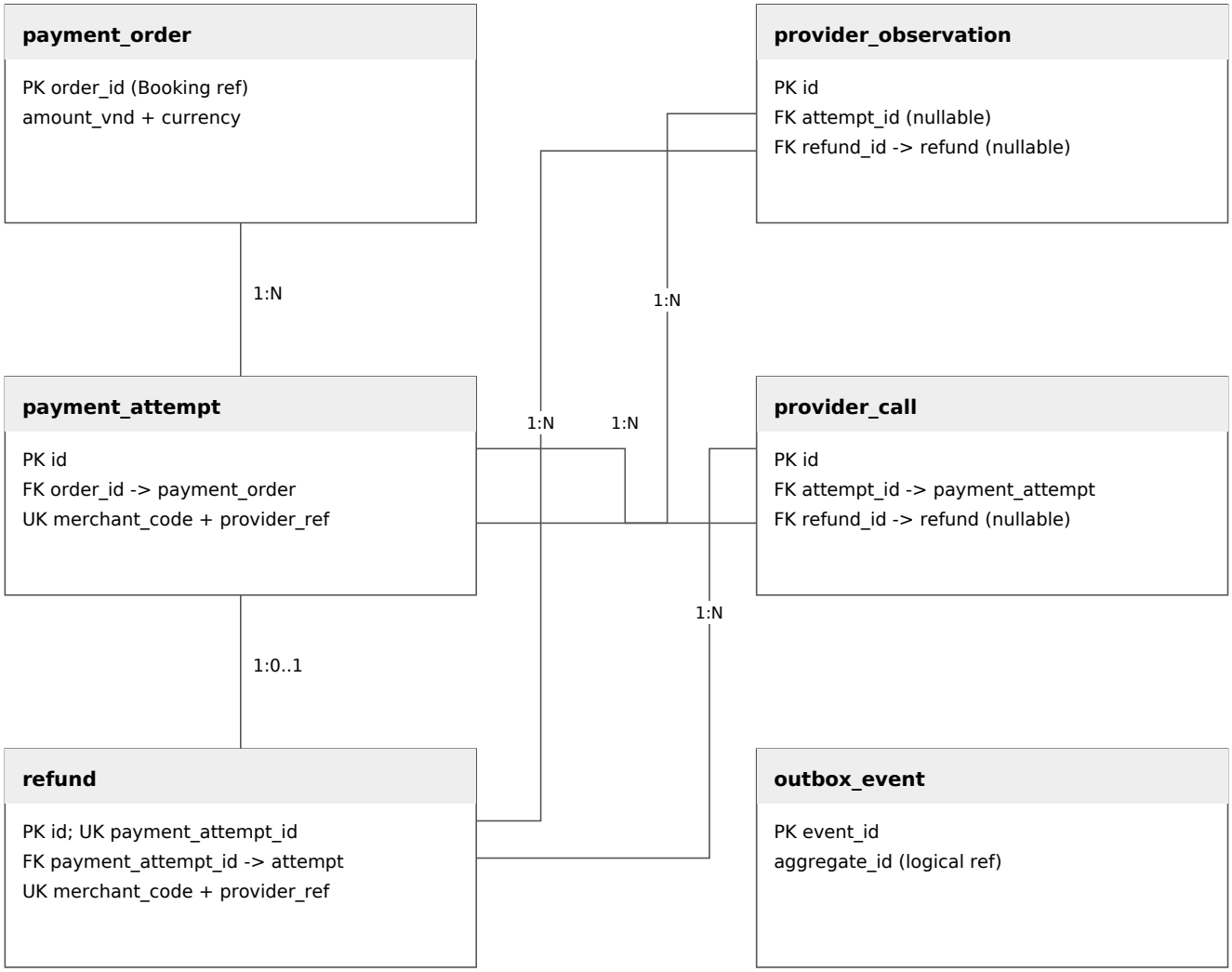
check_in kiểm tra phân công đang active trong transaction; staff_subject không FK đến assignment để không khóa lịch sử khi thu hồi quyền. notification_grant và support_operation là N:1 với booking_order, không liên kết trực tiếp với ticket/check_in.



Ký hiệu: 1 = một; N = nhiều; 0..1 = không có hoặc một. Đường nối là quan hệ FK trong cùng DB; tham chiếu khác service chỉ được ghi bằng ref, không có FK vật lý. Sơ đồ chỉ hiện khóa chính/khóa liên kết; từ điển dữ liệu bên dưới quy định đầy đủ cột.

3.4. ERD Payment

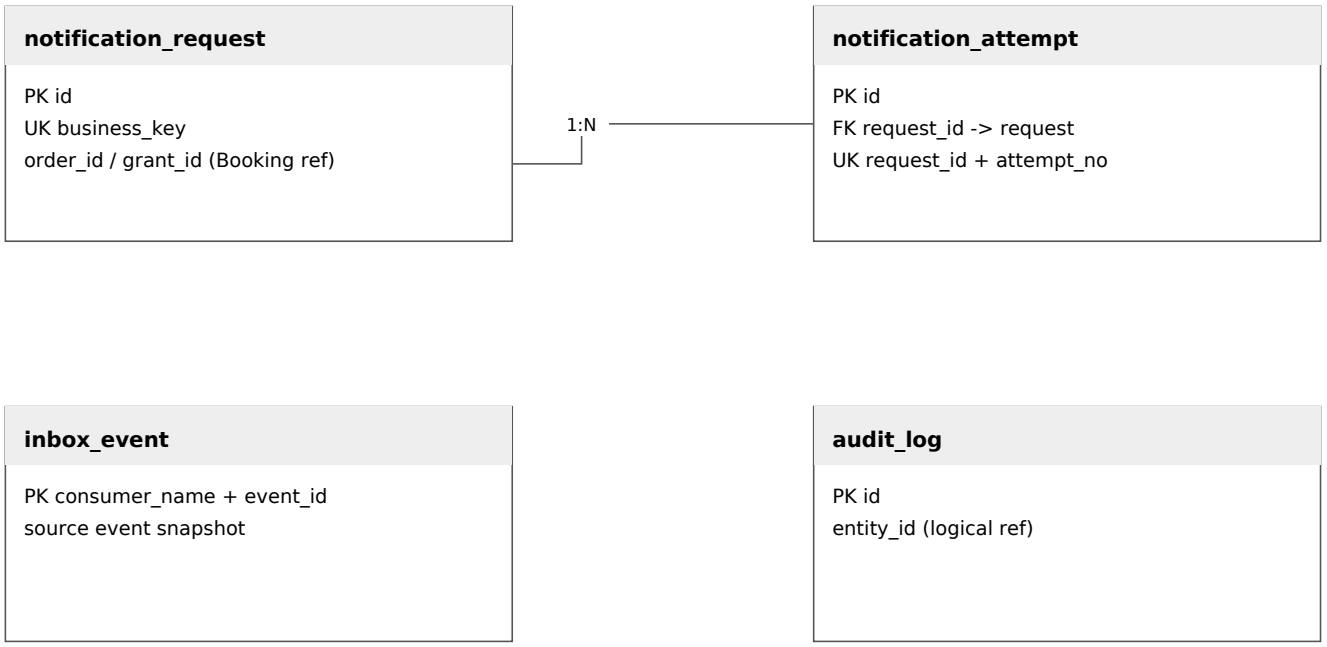
provider_call luôn có attempt_id; provider_observation có thể chưa resolve được attempt. Cả hai có thể thuộc một refund để truy vết khoản hoàn. refund_id có FK; outbox/audit dùng tham chiếu logic đa loại, không FK aggregate. Không FK payment_order sang Booking.



Ký hiệu: 1 = một; N = nhiều; 0..1 = không có hoặc một. Đường nối là quan hệ FK trong cùng DB; tham chiếu khác service chỉ được ghi bằng ref, không có FK vật lý. Sơ đồ chỉ hiện khóa chính/khóa liên kết; từ điển dữ liệu bên dưới quy định đầy đủ cột.

3.5. ERD Notification

Email request được tạo cùng transaction ghi inbox. Không FK sang Booking/Payment. notification_request lưu recipient đã xác minh và metadata, không lưu HTML/ảnh QR. Resend tạo request mới có parent_request_id tham chiếu request gốc.



Ký hiệu: 1 = một; N = nhiều; 0..1 = không có hoặc một. Đường nối là quan hệ FK trong cùng DB; tham chiếu khác service chỉ được ghi bằng ref, không có FK vật lý. Sơ đồ chỉ hiện khóa chính/khóa liên kết; từ điển dữ liệu bên dưới quy định đầy đủ cột.

4. Booking - từ điển dữ liệu

4.1. Phân quyền và Catalog

organizer

platform.organizer - Đơn vị tổ chức; không là tenant DB riêng.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(code). INACTIVE chặn thao tác quản trị mới; không hủy đơn/vé hiện có.

organizer - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
code	varchar(40)	N	UK, mã ổn định
name	varchar(200)	N	Tên hiển thị
status	varchar(16)	N	ACTIVE hoặc INACTIVE

organizer_member

platform.organizer_member - Phạm vi quản lý/tra cứu/báo cáo cho user.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: PK(organizer_id,user_id). Index(user_id,is_active). Role nền từ token kết hợp permission: ORGANIZER cần MANAGER; ADMIN cần SUPPORT_READER hoặc MANAGER để hỗ trợ trong scope. Không có quyền xem QR qua bảng này.

organizer_member - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
organizer_id	uuid	N	PK thành phần; FK organizer(id)
user_id	varchar(128)	N	PK thành phần; Keycloak subject
permission	varchar(24)	N	MANAGER hoặc SUPPORT_READER
is_active	boolean	N	DEFAULT true
granted_by	varchar(128)	N	Admin cấp quyền

venue

catalog.venue - Danh mục địa điểm.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(code). MVP seed danh mục; không mở thêm quy trình quản trị địa điểm. Không sửa địa chỉ làm đổi nội dung suất đã công bố; bản diễn lưu snapshot.

venue - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
code	varchar(40)	N	UK
name	varchar(200)	N	Tên địa điểm
address	varchar(500)	N	Địa chỉ
city	varchar(100)	N	Thành phố
timezone	varchar(64)	N	DEFAULT Asia/Ho_Chi_Minh
is_active	boolean	N	DEFAULT true

venue_gate

catalog.venue_gate - Cổng thuộc một địa điểm.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(venue_id,code), UK(id,venue_id) phục vụ composite FK. Ngừng dùng cổng không xóa lịch sử check-in.

venue_gate - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
venue_id	uuid	N	FK venue(id)
code	varchar(40)	N	Mã cổng
name	varchar(120)	N	Tên cổng
is_active	boolean	N	DEFAULT true

concert

catalog.concert - Aggregate công bố, thuộc organizer.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Index(organizer_id,status,id); UK(id,organizer_id). Publish kiểm nội dung + ít nhất một suất đã cấu hình. Khóa concert khi phát snapshot; tăng search_version cho cả thay đổi suất. HIDDEN không xóa bản sao version.

concert - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
organizer_id	uuid	N	FK organizer(id)
slug	varchar(180)	N	UK, URL ổn định
title	varchar(250)	N	Tên concert
artist_names	jsonb	N	Array chuỗi không rỗng khi publish
description	text	N	Nội dung plain text hoặc HTML đã sanitize
purchase_terms	text	N	Điều kiện mua
status	varchar(16)	N	DRAFT, PUBLISHED, HIDDEN
published_at	timestamptz	Y	Mốc công bố đầu tiên
search_version	bigint	N	DEFAULT 0, CHECK >=0; tăng khi snapshot Search đổi

performance

catalog.performance - Một suất, một khoảng bán.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(concert_id,code); UK(id,concert_id); UK(id,venue_id). CHECK sales_open_at < sales_close_at AND sales_close_at <= starts_at; CHECK checkin_open_at < checkin_close_at. Index(concert_id,starts_at). Không áp đặt cửa sổ vào cổng mặc định; organizer cấu hình. Đổi lịch/venue sau publish bị từ chối trong MVP.

performance - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
concert_id	uuid	N	FK concert(id)
venue_id	uuid	N	FK venue(id)
code	varchar(40)	N	Mã trong concert
starts_at	timestamptz	N	Giờ diễn
sales_open_at	timestamptz	N	Biên mở gồm thời điểm này
sales_close_at	timestamptz	N	Biên đóng không gồm thời điểm này
sales_enabled	boolean	N	DEFAULT false; đóng bán thủ công
checkin_open_at	timestamptz	N	Bắt đầu check-in
checkin_close_at	timestamptz	N	Kết thúc check-in, biên loại trừ
venue_snapshot	jsonb	N	{name,address,city}; bất biến sau publish
inventory_version	bigint	N	DEFAULT 0; tuần tự hóa thay đổi tồn kho

performance_gate

catalog.performance_gate - Cổng được phép quét của một suất.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: PK(performance_id,gate_id). FK(performance_id,venue_id) -> performance(id,venue_id); FK(gate_id,venue_id) -> venue_gate(id,venue_id). Thu hồi cổng bằng is_enabled; check_in vẫn giữ FK.

performance_gate - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
performance_id	uuid	N	PK thành phần
venue_id	uuid	N	Venue dùng để cưỡng chế đúng cổng
gate_id	uuid	N	PK thành phần
is_enabled	boolean	N	DEFAULT true

performance_assignment

platform.performance_assignment - Nhân viên kiểm vé được giao suất.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: PK(performance_id,user_id); index(user_id,is_active). Bắt buộc role CHECKIN_STAFF cộng assignment active. Quản trị cấp quyền xác minh subject tồn tại qua Identity admin adapter; không tạo user trong Booking.

performance_assignment - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
performance_id	uuid	N	PK thành phần; FK performance(id)
user_id	varchar(128)	N	PK thành phần; Keycloak subject
is_active	boolean	N	DEFAULT true
granted_by	varchar(128)	N	Admin thực hiện

4.2. Inventory

performance_section

inventory.performance_section - Khu vực sơ đồ theo từng suất; không thêm reusable venue seat template.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(performance_id,code); UK(id,performance_id). Tên/định danh được snapshot vào order_item.

performance_section - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
performance_id	uuid	N	FK performance(id)
code	varchar(40)	N	Mã khu
name	varchar(120)	N	Tên khu
sort_order	integer	N	DEFAULT 0; CHECK >=0

performance_seat

inventory.performance_seat - Cấu hình ghế và giá hiện hành.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(section_id,performance_id) -> performance_section(id,performance_id). UK(section_id,row_label,seat_number); UK(id,performance_id). Index(performance_id,is_enabled,id). Giá mới chỉ tác động đơn mới; được đổi giá khi ghế HELD nhưng không sửa snapshot. Không di chuyển/đổi nhãn/disable ghế đang HELD hợp lệ hoặc SOLD.

performance_seat - cấu trúc cột

Cột	Kiểu	Null	Ràng buộc / ý nghĩa
id	uuid	N	PK
performance_id	uuid	N	FK performance(id)
section_id	uuid	N	Cùng performance bằng composite FK
row_label	varchar(20)	N	Hàng ghế
seat_number	varchar(20)	N	Số ghế
x	numeric(10,2)	N	Tọa độ sơ đồ, không là tọa độ GPS
y	numeric(10,2)	N	Tọa độ sơ đồ
price_vnd	bigint	N	CHECK >0
is_enabled	boolean	N	DEFAULT true

seat_inventory

inventory.seat_inventory - Một row/ghế, nguồn quyền giữ và bán.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(performance_seat_id,performance_id) -> performance_seat(id,performance_id). FK(current_order_id,performance_id,hold_id) -> booking_order(id,performance_id,hold_id). CHECK: AVAILABLE có cả 4 trường current_order_id/hold_id/expires_at/sold_at null; HELD có order/hold/expiry và sold_at null; SOLD có order/hold/sold_at, expiry null. Index(current_order_id); partial index(expires_at) WHERE status=HELD. Cấp giữ mới được thay row HELD quá hạn, nhưng không sửa đơn cũ dưới khóa ghế.

seat_inventory - cấu trúc cột

Cột	Kiểu	Null	Ràng buộc / ý nghĩa
performance_seat_id	uuid	N	PK; FK performance_seat(id)
performance_id	uuid	N	FK ghép cùng performance_seat
status	varchar(16)	N	AVAILABLE, HELD, SOLD; DEFAULT AVAILABLE
current_order_id	uuid	Y	Đơn giữ hoặc mua hiện tại
hold_id	uuid	Y	Lần giữ hiện tại; giữ lại khi SOLD
expires_at	timestamptz	Y	Có khi HELD; null khi AVAILABLE/SOLD
sold_at	timestamptz	Y	Có khi SOLD

4.3. Ordering

booking_order

ordering.booking_order - Aggregate đơn và thông tin lần giữ.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(concert_id,organizer_id) -> concert(id,organizer_id); FK(performance_id,concert_id) -> performance(id,concert_id). UK(id,performance_id); UK(id,performance_id,hold_id). CHECK expires_at=held_at+5 phút; status/timestamps nhất quán. Index(buyer_subject,created_at DESC,id), (organizer_id,buyer_email_normalized,created_at DESC), partial(expires_at) WHERE status=PENDING_PAYMENT. Dòng đơn và tổng được kiểm bằng use case trong transaction; không cấp DML trực tiếp cho service khác.

booking_order - cấu trúc cột

Cột	Kiểu	Null	Ràng buộc / ý nghĩa
id	uuid	N	PK
order_code	varchar(40)	N	UK, mã tra cứu không tuần tự dễ đoán
buyer_subject	varchar(128)	N	Chủ đơn; Keycloak subject
buyer_email	varchar(320)	N	Email đã xác minh, snapshot khi checkout
buyer_email_normalized	varchar(320)	N	Chuẩn hóa theo Identity cho exact lookup
organizer_id	uuid	N	FK organizer; ghép với concert
concert_id	uuid	N	FK ghép với organizer
performance_id	uuid	N	FK ghép với concert
hold_id	uuid	N	UK; không đổi trên retry
status	varchar(24)	N	PENDING_PAYMENT, CONFIRMED, CANCELLED, EXPIRED
seat_count	smallint	N	CHECK BETWEEN 1 AND 4
subtotal_vnd	bigint	N	CHECK >0
fee_vnd	bigint	N	DEFAULT 0; CHECK =0
total_vnd	bigint	N	CHECK = subtotal_vnd + fee_vnd
currency	char(3)	N	DEFAULT VND; CHECK = VND
held_at	timestamptz	N	Mốc cấp giữ theo DB
expires_at	timestamptz	N	held_at + interval 5 minutes
confirmed_at	timestamptz	Y	Khi CONFIRMED
closed_at	timestamptz	Y	Khi CANCELLED hoặc EXPIRED
close_reason	varchar(64)	Y	USER_CANCELLED, HOLD_EXPIRED, INVENTORY_INCONSISTENCY
confirmed_attempt_id	uuid	Y	Payment ref đã chốt; UK khi khác null
ticket_set_issued_at	timestamptz	Y	Một lần đánh dấu đủ vé
needs_attention	boolean	N	DEFAULT false
attention_reason	varchar(100)	Y	Lý do cần can thiệp
concert_snapshot	jsonb	N	{title,artistNames,startsAt,venue,terms}

order_item

ordering.order_item - Giá và nhãn ghế bất biến của đơn.

Cột chung: created_at (nhóm A).

Khóa / chỉ mục / quy tắc: FK(order_id,performance_id) -> booking_order(id,performance_id); FK(performance_seat_id,performance_id) -> performance_seat(id,performance_id). UK(order_id,performance_seat_id); UK(id,order_id,performance_id,performance_seat_id) cho ticket. Không UK chỉ trên seat_id vì nhiều đơn hết hạn có thể tham chiếu cùng ghế.

order_item - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
order_id	uuid	N	FK booking_order qua cặp performance
performance_id	uuid	N	Cùng suất của đơn và ghế
performance_seat_id	uuid	N	FK performance_seat qua cặp performance
section_label	varchar(120)	N	Snapshot khu
row_label	varchar(20)	N	Snapshot hàng
seat_number	varchar(20)	N	Snapshot số
unit_price_vnd	bigint	N	CHECK >0; quantity luôn 1

booking_payment_view

ordering.booking_payment_view - Projection để điều phối; Payment vẫn là nguồn tiền.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Index(order_id). Cùng transaction: inbox + update projection + workflow_intent. PaymentSuccess phải khớp amount/currency của đơn. Refund event có đủ thông tin khoản thu hoặc được giữ pending để fetch nguồn; không bỏ event chỉ vì projection chưa tồn tại. Không lấy timestamp nhận làm paid_at.

booking_payment_view - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
attempt_id	uuid	N	PK; Payment ref
order_id	uuid	N	FK booking_order(id)
amount_vnd	bigint	N	CHECK >0
currency	char(3)	N	CHECK = VND
payment_status	varchar(16)	N	PROCESSING, VERIFYING, SUCCEEDED, FAILED
paid_at	timestamptz	Y	Thời gian thu nguồn đã xác minh
source_version	bigint	N	Version payment_attempt từ Payment
refund_id	uuid	Y	Reference refund của attempt này
refund_status	varchar(24)	Y	PENDING, SUCCEEDED, NEEDS_SUPPORT
refund_source_version	bigint	Y	Version refund riêng; không so với version attempt
refunded_at	timestamptz	Y	Thời gian hoàn nguồn
last_received_at	timestamptz	N	Thời gian consumer nhận

workflow_intent

ordering.workflow_intent - Hàng đợi chuyển tiếp bền vững tới Temporal, không là job engine mới.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Index(state,next_attempt_at). START ghi cùng đơn; signal ghi cùng inbox. START không phụ thuộc Kafka. Dispatcher start-or-signal theo workflow_id xác định; crash sau signal cho phép lặp, activity kiểm trạng thái DB. Lease hết không chứng minh side effect chưa chạy.

workflow_intent - cấu trúc cột

Cột	Kiểu	Null	Ràng buộc / ý nghĩa
id	uuid	N	PK
order_id	uuid	N	FK booking_order(id)
kind	varchar(32)	N	START_ORDER, PAYMENT_CHANGED, REFUND_CHANGED, SUPPORT_RETRY
business_key	varchar(160)	N	UK; order hoặc event/operation ID
workflow_id	varchar(160)	N	order:{orderId}
payload	jsonb	N	Reference, version; không QR/PII không cần thiết
state	varchar(16)	N	PENDING, IN_FLIGHT, DELIVERED, ATTENTION
attempt_count	integer	N	DEFAULT 0, CHECK >=0
next_attempt_at	timestamptz	N	Lịch thử
lease_token	uuid	Y	Fencing token khi claim
lease_until	timestamptz	Y	Hạn claim
last_error_code	varchar(80)	Y	Lỗi đã che bí mật
delivered_at	timestamptz	Y	Khi Temporal nhận thành công

4.4. Ticketing và hỗ trợ

ticket

ticketing.ticket - Một vé/ghế đã bán; không lưu ảnh QR hoặc PDF.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(order_item_id,order_id,performance_id,performance_seat_id) -> order_item(id,order_id,performance_id,performance_seat_id); UK(id,performance_id). CHECK digest dài 32 byte, nonce 12 byte; CHECK ISSUED/used_at null hoặc USED/used_at not null. Owner lấy bằng join booking_order, không thêm buyer copy ở ticket. AAD gồm ticket_id; dùng thư viện AEAD đã kiểm chứng. Rotate khóa chỉ re-encrypt token cũ, không phát QR mới.

ticket - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
ticket_code	varchar(40)	N	UK, mã tra cứu không phải token vào cổng
order_item_id	uuid	N	UK; FK ghép tới order_item
order_id	uuid	N	Một phần composite FK
performance_id	uuid	N	Một phần composite FK
performance_seat_id	uuid	N	UK; một ghế bán chỉ có một vé
status	varchar(16)	N	ISSUED hoặc USED
issued_at	timestamptz	N	Mốc phát hành lần đầu
used_at	timestamptz	Y	Có khi USED
qr_digest	bytea	N	UK; SHA-256 token 32 byte ngẫu nhiên
qr_ciphertext	bytea	N	Token mã hóa AEAD, gồm authentication tag
qr_nonce	bytea	N	Nonce ngẫu nhiên 12 byte
qr_key_id	varchar(64)	N	Phiên bản khóa ngoài database

check_in

ticketing.check_in - Bản ghi một lần sử dụng thành công, không dùng cho mọi lần quét lỗi.

Cột chung: created_at (nhóm A).

Khóa / chỉ mục / quy tắc: FK(ticket_id,performance_id) -> ticket(id,performance_id); FK(performance_id,gate_id) -> performance_gate(performance_id,gate_id). Index(performance_id,checked_in_at); index(staff_subject,checked_in_at). UPDATE ticket và INSERT check_in + audit/outbox cùng transaction. Scan thất bại vào audit đã che token.

check_in - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
ticket_id	uuid	N	PK; FK ticket qua performance
performance_id	uuid	N	Cùng suất vé/cổng
gate_id	uuid	N	Cổng của suất
staff_subject	varchar(128)	N	Người quét
checked_in_at	timestamptz	N	Mốc DB
request_id	uuid	N	Correlation của lần thành công

notification_grant

ticketing.notification_grant - Cho Notification lấy QR chỉ cho thông báo hợp lệ.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Grant là reference, không bearer credential. API cần service identity Notification + grant ACTIVE; đối chiếu request/order/owner. Cho retry nhiều lần cùng grant; không cấp toàn quyền lấy QR theo orderId tùy ý. Không trả token qua API hỗ trợ.

notification_grant - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
order_id	uuid	N	FK booking_order(id)
notification_key	varchar(160)	N	UK; TICKET_SET:{orderId} hoặc RESEND:{operationId}
owner_subject	varchar(128)	N	Chủ đơn snapshot
recipient_email	varchar(320)	N	Email đã xác minh của đơn
purpose	varchar(24)	N	TICKET_SET hoặc TICKET_RESEND
state	varchar(16)	N	ACTIVE hoặc REVOKED
requested_by	varchar(128)	Y	Admin khi resend

support_operation

platform.support_operation - Theo dõi can thiệp đã tiếp nhận, không điều phối mọi nghiệp vụ.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Index(order_id,created_at); cùng operationId luôn gọi nguồn với cùng key. Outbox/intent cùng transaction ACCEPTED. COMPLETED là tác vụ hỗ trợ xong theo loại, không đồng nghĩa refund đã thành công nếu chỉ yêu cầu query. Phản hồi kèm trạng thái nguồn hiện tại.

support_operation - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
order_id	uuid	N	FK booking_order(id)
target_service	varchar(24)	N	BOOKING, PAYMENT, NOTIFICATION
target_id	varchar(160)	N	Order/attempt/refund/request ref
action	varchar(32)	N	VERIFY_PAYMENT, RETRY_REFUND, ISSUE_MISSING_TICKETS, RESEND_EMAIL
actor_subject	varchar(128)	N	Admin
reason	varchar(500)	N	Lý do bắt buộc
state	varchar(24)	N	ACCEPTED, RUNNING, COMPLETED, REJECTED, NEEDS_SUPPORT
result_ref	varchar(160)	Y	ID kết quả nguồn
last_error_code	varchar(80)	Y	Lỗi xử lý

5. Payment - từ điển dữ liệu

payment_order

Payment / public.payment_order - Bản sao bất biến của tổng tiền đơn, đồng thời là row khóa để tạo attempt.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(successful_attempt_id,order_id) -> payment_attempt(id,order_id); index(organizer_id,created_at). INSERT ON CONFLICT rồi SELECT FOR UPDATE(order_id) để serialize. Snapshot khác dữ liệu đã lưu trả 409. Luôn kiểm DB Booking qua hợp đồng đã xác thực trước khi bắt đầu lần thu mới; kiểm lần nữa hạn bằng DB Payment, không giữ lock trong HTTP call.

payment_order - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
order_id	uuid	N	PK; Booking reference, không FK xuyên DB
organizer_id	uuid	N	Booking scope ref
performance_id	uuid	N	Booking scope ref
buyer_subject	varchar(128)	N	Chủ đơn nguồn
amount_vnd	bigint	N	CHECK >0
currency	char(3)	N	CHECK = VND
hold_expires_at	timestamptz	N	Hạn nguồn, không được gia hạn
successful_attempt_id	uuid	Y	Khoản thu đầu tiên được chọn; FK bổ sung sau tạo attempt
next_attempt_no	integer	N	DEFAULT 1; CHECK >=1
needs_attention	boolean	N	DEFAULT false

payment_attempt

Payment / public.payment_attempt - Một lần thử thu với provider reference ổn định.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: UK(order_id,attempt_no); UK(id,order_id); UK(merchant_code,provider_ref); unique partial(merchant_code,provider_transaction_id) WHERE provider_transaction_id IS NOT NULL. Unique partial(order_id) WHERE status IN (PROCESSING,VERIFYING). Index(status,next_check_at). Không tạo lần mới nếu successful_attempt_id đã có; kiểm dưới khóa payment_order. Giữ lịch sử SUCCEEDED ngay cả khi đã refund.

payment_attempt - cấu trúc cột

Cột	Kiểu	Null	Ràng buộc / ý nghĩa
id	uuid	N	PK
order_id	uuid	N	FK payment_order(order_id)
attempt_no	integer	N	Số lần của đơn
merchant_code	varchar(64)	N	Merchant cấu hình
provider_ref	varchar(100)	N	vnp_TxnRef do adapter tạo, không đổi
provider_transaction_id	varchar(100)	Y	ID giao dịch do provider trả
status	varchar(16)	N	PROCESSING, VERIFYING, SUCCEEDED, FAILED
paid_at	timestamptz	Y	Mốc thu đã xác minh
verified_at	timestamptz	Y	Mốc hệ thống xác minh
failure_code	varchar(80)	Y	Chỉ khi có thất bại rõ ràng
payment_url_ciphertext	bytea	Y	Envelope mã hóa URL gồm nonce/keyId/tag; không log
provider_deadline	timestamptz	Y	Hạn adapter nếu hỗ trợ; không thay hạn hold
next_check_at	timestamptz	Y	Lịch query
check_count	integer	N	DEFAULT 0, CHECK >=0
lease_token	uuid	Y	Claim/fencing token
lease_until	timestamptz	Y	Hạn xử lý
needs_attention	boolean	N	DEFAULT false

Partial unique chặn hai attempt đang mở; khóa payment_order + successful_attempt_id chặn attempt mới sau thu thành công. Không dùng unique(order_id) trên mọi khoản thu SUCCEEDED vì phải giữ được bằng chứng một khoản thu bất thường thứ hai nếu provider sửa kết quả FAILED cũ sau khi đã tạo attempt mới. Trường hợp bất thường này được đối chiếu, audit và hoàn khoản thừa; không phủ nhận tiền thực đã thu chỉ để qua constraint.

refund

Payment / public.refund - Một nghĩa vụ hoàn toàn bộ cho mỗi khoản thu.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: FK(payment_attempt_id,order_id) -> payment_attempt(id,order_id). UK(merchant_code,provider_ref); index(status,next_check_at). CHECK SUCCEEDED có refunded_at/verified_at; amount=full amount kiểm dưới lock payment_order + attempt. Retry không đổi id/provider_ref/amount. ACCEPTED không tự thành SUCCEEDED.

refund - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
payment_attempt_id	uuid	N	UK; FK payment_attempt(id)
order_id	uuid	N	Cặp FK với attempt
amount_vnd	bigint	N	CHECK >0; bằng payment_order.amount_vnd
currency	char(3)	N	CHECK = VND
reason	varchar(64)	N	ORDER_EXPIRED, ORDER_CANCELLED, INVENTORY_INCONSISTENCY, EXCESS_CHARGE
status	varchar(24)	N	PENDING, SUCCEEDED, NEEDS_SUPPORT
merchant_code	varchar(64)	N	Merchant nguồn
provider_ref	varchar(100)	N	Reference refund ổn định
provider_refund_id	varchar(100)	Y	ID provider nếu có
provider_state	varchar(24)	N	NOT_SENT, UNKNOWN, ACCEPTED, CONFIRMED, REJECTED
refunded_at	timestamptz	Y	Thời gian hoàn thực tế khi SUCCEEDED
verified_at	timestamptz	Y	Thời gian xác minh
next_check_at	timestamptz	Y	Lịch query/đối chiếu
attempt_count	integer	N	DEFAULT 0, CHECK >=0
lease_token	uuid	Y	Claim/fencing token
lease_until	timestamptz	Y	Hạn claim
last_error_code	varchar(80)	Y	Lỗi đã che thông tin

provider_call

Payment / public.provider_call - Lịch sử từng lần gọi ra ngoài và đối chiếu.

Cột chung: không bổ sung ngoài các cột đã liệt kê.

Khóa / chỉ mục / quy tắc: Index(attempt_id,started_at); index(refund_id,started_at). STARTED được commit trước I/O. Row có thể hoàn tất sau call, không append-only tuyệt đối. refund/attempt cùng khoản thu được kiểm use case. Không lưu body chứa chữ ký/PII tùy ý. Query refund chỉ được bật khi contract merchant đã kiểm chứng.

provider_call - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
attempt_id	uuid	N	FK payment_attempt(id)
refund_id	uuid	Y	FK refund(id)
operation	varchar(24)	N	BUILD_URL, QUERY_PAYMENT, SUBMIT_REFUND, QUERY_REFUND
request_ref	varchar(100)	N	Reference cụ thể theo hợp đồng adapter
request_hash	char(64)	N	Hash canonical request đã bỏ secret
started_at	timestamptz	N	Trước I/O
finished_at	timestamptz	Y	Sau I/O; null nếu crash
result	varchar(16)	N	STARTED, RESPONSE, TIMEOUT, ERROR
response_code	varchar(80)	Y	Mã đã sanitize
response_summary	jsonb	Y	Whitelist trạng thái/ref/amount; không secret

provider_observation

Payment / public.provider_observation - Bằng chứng IPN/query, kể cả mâu thuẫn.

Cột chung: created_at (nhóm A).

Khóa / chỉ mục / quy tắc: UK(fingerprint); index(attempt_id,received_at); index(handling_result,received_at). Request giả mạo bị rate limit; không ghi vô hạn payload. Callback trùng không tạo tác động tiền, acknowledgement trả kết quả phù hợp hợp đồng provider.

provider_observation - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
attempt_id	uuid	Y	FK payment_attempt(id); null nếu không resolve
refund_id	uuid	Y	FK refund(id)
source	varchar(24)	N	IPN, PAYMENT_QUERY, REFUND_QUERY
fingerprint	char(64)	N	UK; hash dữ liệu canonical + merchant + source
provider_ref	varchar(100)	Y	Reference đã giới hạn độ dài
signature_valid	boolean	N	Kết quả xác minh
amount_vnd	bigint	Y	Giá trị parse được, chưa là bằng chứng thu
provider_status	varchar(80)	Y	Trạng thái gốc whitelist
provider_business_at	timestamptz	Y	Mốc giao dịch nguồn
received_at	timestamptz	N	Mốc nhận
handling_result	varchar(24)	N	APPLIED, DUPLICATE, REJECTED, CONFLICT, UNRESOLVED
safe_payload	jsonb	N	Chỉ trường cho phép; bỏ signature, PII, secret

shedlock

Payment / public.shedlock - Bảng khóa job reconciliation, không là khóa tiền.

Cột chung: không bổ sung ngoài các cột đã liệt kê.

Khóa / chỉ mục / quy tắc: Schema adapter kiểm lại theo phiên bản dependency khi init. Thời hạn lock và cơ chế kéo dài cấu hình theo batch. Mất lease vẫn phải an toàn nhờ row claim + reference; không retry refund mù.

shedlock - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
name	varchar(64)	N	PK; ví dụ paymentReconcile hoặc refundReconcile
lock_until	timestamp(3)	N	UTC; theo adapter ShedLock được pin
locked_at	timestamp(3)	N	UTC
locked_by	varchar(255)	N	Worker/instance

6. Notification - từ điển dữ liệu

notification_request

Notification / public.notification_request - Một thông báo nghiệp vụ hoặc resend chủ đích.

Cột chung: created_at, updated_at, version (nhóm M).

Khóa / chỉ mục / quy tắc: Index(status,next_attempt_at); index(order_id,created_at). Không persist nội dung HTML/PNG/QR. Template render trong memory; recipient không nhận từ request tùy ý của admin. Refund event chỉ có reference; fetch recipient từ Booking bằng service API có quyền và retry nếu tạm lỗi.

notification_request - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
business_key	varchar(160)	N	UK; TICKET_SET:{orderId}, REFUND:{refundId}, RESEND:{operationId}
type	varchar(24)	N	TICKET_SET, REFUND_SUCCEEDED
order_id	uuid	N	Booking ref
refund_id	uuid	Y	Payment ref
booking_grant_id	uuid	Y	Bắt buộc cho email chứa QR
recipient_email	varchar(320)	N	Từ grant/snapshot đã xác minh
recipient_subject	varchar(128)	N	Chủ đơn
parent_request_id	uuid	Y	FK notification_request(id), khi resend
template_version	varchar(40)	N	Version template, giữ cố định khi retry
status	varchar(24)	N	PENDING, SENDING, SENT, RETRY_REQUIRED, NEEDS_SUPPORT
attempt_count	integer	N	DEFAULT 0
next_attempt_at	timestamptz	N	Lịch gửi
lease_token	uuid	Y	Token claim
lease_until	timestamptz	Y	Hạn claim
sent_at	timestamptz	Y	SMTP accept được ghi nhận
last_error_code	varchar(80)	Y	Lỗi đã che thông tin

notification_attempt

Notification / public.notification_attempt - Lịch sử gửi, bao gồm chưa rõ SMTP đã nhận hay chưa.

Cột chung: không bổ sung ngoài các cột đã liệt kê.

Khóa / chỉ mục / quy tắc: UK(request_id,attempt_no). SENT nghĩa là SMTP đã accept, không khẳng định người mua đã đọc. Crash/mất response dẫn UNKNOWN; retry có thể gửi lặp dù giữ Message-ID. Lịch sử hỗ trợ chỉ trả metadata đã che recipient.

notification_attempt - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
request_id	uuid	N	FK notification_request(id)
attempt_no	integer	N	CHECK >0
started_at	timestamptz	N	Trước SMTP
finished_at	timestamptz	Y	Sau SMTP
result	varchar(16)	N	STARTED, SENT, FAILED, UNKNOWN
message_id	varchar(200)	N	Message-ID ổn định theo request
provider_reference	varchar(200)	Y	SMTP/provider response ref
error_code	varchar(80)	Y	Mã lỗi đã sanitize

7. Search - mapping và đồng bộ

Không thêm PostgreSQL cho Search. Elasticsearch index concert_v1 có alias concert_read và concert_write. Một document/concert; trường performances dùng nested để tất cả điều kiện địa điểm/ngày khớp trên cùng một suất. is_public=false là tombstone, vẫn giữ source_version để bỏ snapshot cũ.

Trường	**Kiểu mapping**	**Ý nghĩa**
_id / concert_id	keyword	ID concert; _id=concert_id
organizer_id	keyword	Phạm vi nguồn
source_version	long	concert.search_version
title / artist_names	text + keyword subfield	Analyzer chuẩn + lowercase + asciifolding; thử bộ tên tiếng Việt và đ/Đ khi khóa mapping.
status / is_public	keyword / boolean	DRAFT/PUBLISHED/HIDDEN; false khi không public
performances	nested	Mỗi phần tử chứa performance_id keyword, venue_id keyword, venue_name text, city keyword, starts_at date, sales_open_at date, sales_close_at date, sales_enabled boolean.
updated_at	date	Mốc thay đổi nguồn
indexed_at	date	Mốc nhận tại Search

Index: truy vấn bool filter is_public=true + nested performance; sort min(starts_at) của suất khớp, rồi concert_id. q áp dụng title/artist_names/venue_name theo hợp đồng. Không lọc giá. Phân trang search_after có cursor; scan bù candidate bị ẩn có giới hạn, không hứa số kết quả bằng pageSize nếu nguồn loại dữ liệu.

Consumer áp dụng snapshot khi incoming source_version > stored version bằng thao tác nguyên tử tại Elasticsearch; bằng version thì no-op, version nhỏ bỏ qua. Sau write thành công mới commit Kafka offset. Không cần inbox SQL: version + document ID bảo vệ replay. Không dùng thứ tự Kafka làm bằng chứng duy nhất.

Trước khi trả public DTO: batch validate ID và trạng thái với Booking; dùng trường public Booking trả để tránh lộ nội dung cũ. Booking unavailable thì không trả nội dung chưa xác minh. Reindex: tạo generation mới, nạp snapshot kèm version qua API Booking, replay delta từ offset bắt đầu đã ghi nhận, bắt kịp rồi đổi alias. Giữ tombstone cả khi snapshot hiện không public; không hard-delete version.

8. Report - collection, raw dataset và batch

MongoDB dùng BSON UUID cho ID, Date UTC, NumberLong cho số tiền/counter; không double cho VND. Bật JSON schema validation cho required/type/status. Quan hệ là reference ứng dụng, không FK. MinIO chỉ giữ raw facts không chứa QR, OTP hoặc email không cần thiết.

Collection	Khóa / chỉ mục	Fields bắt buộc và nullable
raw_event	_id=eventId unique; unique(source,topic,partition,offset); index(export_state,received_at)	source, event_type, schema_version, aggregate_id, aggregate_version, occurred_at, received_at, topic, partition, offset, payload(object), payload_hash, export_state(PENDING/CLAIMED/EXPORTED); manifest_id?, lease_token?, lease_until?
raw_manifest	_id=manifestId; unique(object_key); index(state,created_at)	state(PREPARED/READY/FAILED), event_ids(array UUID), object_key, sha256?, event_count, source_offsets(object), min_business_at, max_business_at, created_at, completed_at?, error_code?
report_run	_id=runId; unique(input_hash,logic_version) cho lần chạy tái lập	state(BUILDING/VALIDATED/ACTIVE/FAILED), manifest_ids[], input_hash, logic_version, source_watermarks[], business_cutoff, started_at, completed_at?, row_count?, checksum?, error_code?
daily_metric	unique(run_id,organizer_id,performance_id,business_date); index(run_id,organizer_id,business_date)	run_id, organizer_id, concert_id, performance_id, business_date(YYYY-MM-DD), tickets_issued, checked_in, collected_vnd, refunded_vnd, net_vnd
occupancy_snapshot	unique(run_id,performance_id,as_of)	run_id, organizer_id, performance_id, as_of, source_inventory_version, sold_count, enabled_seat_count; ratio null khi mẫu số=0
report_detail	unique(run_id,fact_type,business_key); index(run_id,organizer_id,order_id)	run_id, fact_type, business_key, organizer_id, performance_id, order_id?, attempt_id?, refund_id?, ticket_id?, occurred_at, amount_vnd?, source_event_id, status
report_pointer	_id=default	active_run_id, generation(NumberLong), activated_at; cập nhật CAS generation
source_checkpoint	_id=source:topic:partition	contiguous_ingested_offset, contiguous_exported_offset, updated_at; không coi max offset là contiguous nếu có gap

8.1. Quan hệ và giao thức ghi

Từ	Đến	Cardinality / cách bảo vệ
raw_manifest.event_ids	raw_event._id	1:N; exporter claim bền vững một tập cố định.
report_run.manifest_ids	raw_manifest._id	N:N; chỉ đọc READY và checksum hợp lệ.
daily_metric / occupancy / detail	report_run._id	N:1; API chỉ đọc run đã activate.
report_pointer.active_run_id	report_run._id	1:1 tại thời điểm đọc; pointer đổi một lần sau validation.

1. Ingest insert raw_event theo eventId, duplicate cùng hash là no-op; trùng ID khác hash đưa quarantine/alert. Chỉ commit Kafka offset sau durable insert hoặc xác nhận duplicate cùng hash; source_checkpoint vẫn tiến qua offset của bản lặp dù raw_event chỉ lưu vị trí nhận đầu tiên. Không cần

transaction với Kafka.

2. Exporter tạo manifest PREPARED, lưu event_ids cố định rồi CAS từng raw_event sang CLAIMED cùng manifest. Chỉ xuất sau khi tất cả event được claim đúng manifest. Nếu một event bị claim bởi manifest khác: đánh manifest FAILED, release các claim của chính manifest bằng CAS rồi chọn batch mới; không sửa danh sách đã chuẩn bị. Recovery resume PREPARED hoặc rollback claim, luôn kiểm manifest_id. Không xóa raw khi chưa READY.

3. Ghi JSONL hoặc Parquet vào raw/{source}/{manifestId}.parquet với key ổn định và checksum. Timeout đọc metadata/checksum object trước retry. Sau object hoàn tất cập nhật READY, rồi đánh dấu các event EXPORTED. Crash được resume từ manifest, không tạo batch ngẫu nhiên khác cho cùng claim.

4. Spark nhận runId + manifest set cố định + business cutoff + logic version. Dedup theo eventId, sau đó theo business key và version từng aggregate. Không sum mọi state-change event như giao dịch mới.

5. Upsert toàn bộ output vào run mới; validate tổng, số dòng và checksum. Đặt run VALIDATED rồi CAS report_pointer sang run mới. API đọc pointer một lần/response, luôn query cùng runId. Nếu crash trước đổi pointer thì API vẫn đọc run cũ; sau pointer đổi run có thể được sửa ACTIVE bằng recovery.

8.2. Công thức và lịch sử lấp đầy

Tiền thu chỉ lấy một fact SUCCEEDED/attemptId; tiền hoàn một fact SUCCEEDED/refundId. Vé một fact/ticketId ở issued_at; check-in một fact/ticketId ở checked_in_at. Nhóm ngày từ thời gian nghiệp vụ theo Asia/Ho_Chi_Minh. net_vnd=collected_vnd-refunded_vnd, được âm.

Mỗi thay đổi ghế cấu hình hoặc chuyển tồn kho trong Booking phát InventoryChanged gồm performance_id, inventory_version, effective_at và danh sách ghế thay đổi (seatId, enabled, statusAfter). Cùng transaction tăng performance.inventory_version. Sự kiện SeatMapInitialized là baseline đầy đủ theo suất. Report tái dựng theo version và cutoff để tính SOLD/enabled cùng mốc, không dùng cấu hình hiện tại cho mẫu số lịch sử. Nếu phát hiện version gap, chưa đánh dấu snapshot hoàn chỉnh; yêu cầu replay/baseline nguồn. Giá không cần cho lấp đầy.

SOLD không được disable. Gộp nhiều suất bằng tổng sold_count / tổng enabled_seat_count, không lấy trung bình tỷ lệ. Snapshot cuối ngày có thể được hiệu chỉnh bởi run mới khi dữ liệu đến trễ. Trả watermark riêng từng nguồn/partition, business_cutoff và runId; không dùng started_at làm mốc dữ liệu đã đủ.

9. Bảng kỹ thuật dùng chung và Identity

Các bảng dưới đây là cấu trúc chung, mỗi service tạo bản riêng và chỉ ghi DB mình. Booking: idempotency_record, outbox_event, inbox_event, audit_log. Payment: idempotency_record, outbox_event, audit_log; không cần inbox khi chỉ nhận REST và IPN. Notification: inbox_event, audit_log; resend được dedup bằng business_key. Không tạo các bảng SQL này trong Search/Report.

idempotency_record

Booking/Payment - Lưu kết quả thao tác ghi bền vững.

Cột chung: created_at (nhóm A).

Khóa / chỉ mục / quy tắc: PK(actor_key,operation,idempotency_key). Không xóa TTL trong MVP. Cùng key khác hash => 409. Trước khóa nghiệp vụ: lấy advisory transaction lock theo hash64(actor,operation,key), đọc lại key; cùng key/hash trả kết quả cũ. INSERT bản ghi hoàn tất trong transaction nghiệp vụ; PK vẫn là rào chắn cuối. Hash collision chỉ làm serialize dư, không thay phép so key đầy đủ. Không commit row IN_PROGRESS mồ côi. Lỗi trước commit không chiếm key vĩnh viễn.

idempotency_record - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
actor_key	varchar(160)	N	PK thành phần; subject hoặc service identity
operation	varchar(64)	N	PK thành phần; CHECKOUT, START_PAYMENT, CANCEL_ORDER, SUPPORT_ACTION, REQUEST_REFUND
idempotency_key	varchar(128)	N	PK thành phần; key client/service
request_hash	char(64)	N	SHA-256 canonical payload + resource scope
resource_id	uuid	N	Đơn/attempt/refund/operation tạo hoặc lấy
response_code	smallint	N	HTTP outcome đã commit
response_body	jsonb	N	DTO tối thiểu; không QR/URL bí mật
completed_at	timestamptz	N	Mốc commit logic

outbox_event

Booking/Payment - Event sẽ publish Kafka trong cùng transaction nghiệp vụ.

Cột chung: created_at (nhóm A).

Khóa / chỉ mục / quy tắc: UK(aggregate_type,aggregate_id,aggregate_version,stream_seq). Index(state,next_attempt_at). Claim FOR UPDATE SKIP LOCKED; commit claim rồi I/O; ack update theo lease_token. Có thể publish lặp; consumer dedup/version. Không giả định relay nhiều worker luôn giữ thứ tự; projection dùng snapshot/version.

outbox_event - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
event_id	uuid	N	PK
aggregate_type	varchar(40)	N	Concert, Inventory, Order, Ticket, Payment, Refund
aggregate_id	uuid	N	Logical ref
aggregate_version	bigint	N	Version nguồn
schema_version	integer	N	DEFAULT 1
stream_seq	smallint	N	Thứ tự event cùng version nếu cần
event_type	varchar(64)	N	Tên event contract
topic	varchar(120)	N	Topic đích
partition_key	varchar(128)	N	Aggregate ID phù hợp
envelope	jsonb	N	Envelope + payload đã whitelist
occurred_at	timestamptz	N	Mốc nghiệp vụ
state	varchar(16)	N	PENDING, IN_FLIGHT, PUBLISHED, ATTENTION
attempt_count	integer	N	DEFAULT 0
next_attempt_at	timestamptz	N	Lịch relay
lease_token	uuid	Y	Fencing claim
lease_until	timestamptz	Y	Hạn claim
published_at	timestamptz	Y	Sau broker ack
last_error_code	varchar(80)	Y	Lỗi đã sanitize

inbox_event

Booking/Notification - Chống xử lý lặp, lưu event chưa hiểu/không hợp lệ để điều tra.

Cột chung: không bổ sung ngoài các cột đã liệt kê.

Khóa / chỉ mục / quy tắc: PK(consumer_name,event_id); index(state,received_at). Inbox + projection/notification/intent commit chung. Cùng eventId khác hash không coi là duplicate bình thường. PENDING cần worker retry; commit offset chỉ khi event đã được xử lý hoặc lưu bền vững để xử lý tiếp.

inbox_event - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
consumer_name	varchar(80)	N	PK thành phần
event_id	uuid	N	PK thành phần
source	varchar(40)	N	Producer service
aggregate_id	uuid	N	Reference nguồn
aggregate_version	bigint	N	Version nguồn
payload_hash	char(64)	N	Hash canonical event
envelope	jsonb	N	Payload không QR/secret
state	varchar(16)	N	APPLIED, PENDING, QUARANTINED
received_at	timestamptz	N	Mốc nhận
processed_at	timestamptz	Y	Mốc áp dụng
error_code	varchar(80)	Y	Lý do chưa áp dụng

audit_log

Booking/Payment/Notification - Audit riêng theo nguồn, append-only.

Cột chung: không bổ sung ngoài các cột đã liệt kê.

Khóa / chỉ mục / quy tắc: Index(entity_type,entity_id,occurred_at); index(actor_subject,occurred_at); index(organizer_id,occurred_at). App role chỉ INSERT/SELECT audit; không UPDATE/DELETE. Audit thành công cùng transaction; audit từ chối dùng transaction riêng sau rollback. Không ghi request body chứa QR, password, OTP hay token.

audit_log - cấu trúc cột			
Cột	**Kiểu**	**Null**	**Ràng buộc / ý nghĩa**
id	uuid	N	PK
actor_type	varchar(16)	N	USER, SERVICE, SYSTEM
actor_subject	varchar(160)	N	User/service ID
action	varchar(80)	N	Sửa giá, đóng bán, check-in, refund, cấp quyền, hỗ trợ...
entity_type	varchar(40)	N	Loại đối tượng
entity_id	varchar(160)	N	Logical ref
organizer_id	uuid	Y	Scope nguồn
order_id	uuid	Y	Reference tra chuỗi
outcome	varchar(16)	N	SUCCESS, REJECTED, ERROR
reason_code	varchar(80)	Y	Lý do
before_data	jsonb	Y	Whitelist; giá/quyền/trạng thái cũ
after_data	jsonb	Y	Whitelist; giá/quyền/trạng thái mới
correlation_id	uuid	N	Liên kết request/event
operation_id	uuid	Y	Can thiệp hỗ trợ
occurred_at	timestamptz	N	Mốc DB

9.1. Identity và SMS OTP

Không tự thiết kế lại bảng user/credential/session của Keycloak. Schema nội bộ Keycloak/Temporal và Liquibase DATABASECHANGELOG/LOCK thuộc sản phẩm; không đưa migration nghiệp vụ vào đó. LLD quy định dữ liệu extension và claim contract sau:

Dữ liệu	Thiết kế
User / claim	sub ổn định; username=phone E.164; email duy nhất; email_verified và custom phone_verified. Checkout bắt buộc cả hai true. Tên hiển thị không là username.
Role nền	BUYER, ORGANIZER, CHECKIN_STAFF, ADMIN; quyền trên từng organizer/performance luôn tra Booking. Token role không chứa danh sách QR hoặc quyền bypass.
OTP store	Redis namespace identity riêng cho Keycloak extension; không phải Notification. Mất Redis hủy challenge và yêu cầu gửi lại, không được tự verified.
challenge:{session}:{purpose}	Hash: challenge_id UUID, subject?, phone_e164, purpose=VERIFY_PHONE, otp_mac, mac_key_id, issued_at, expires_at, attempts_remaining, generation, state=PENDING/SENT/CONSUMED/LOCKED. TTL bằng thời hạn challenge.
Chống đoán / gửi lại	Mã 6 số; cấu hình khởi điểm TTL 5 phút, tối đa 5 lần sai, resend cooldown 60 giây. Đây là baseline thử nghiệm LLD, không SLA. Limit theo phone HMAC/IP/session; budget SMS bắt buộc cấu hình trước bật gửi thật.
Atomicity	Redis script kiểm challenge_id/generation/state/expiry và giảm attempts hoặc consume một lần nguyên tử. Resend thay generation và vô hiệu mã cũ. OTP MAC=HMAC với secret ngoài Redis; không hash mã 6 số không có secret.
SMS timeout / crash	Không đánh verified khi provider gửi thành công. Chỉ xác minh mã đúng từ challenge hợp lệ. Timeout gửi vẫn giữ challenge và cooldown; khi đã có SMS khách có thể nhập mã. Không retry gửi vô hạn.
Hoàn tất xác minh	Consume challenge trước khi đánh phone_verified; crash ở giữa có thể buộc xác minh lại, không bỏ qua. Lưu Keycloak user/admin event đã che mã. Phone/username không tự sửa trong MVP.
Khôi phục / phiên	Reset password qua email đã xác minh; không OTP mỗi lần login. OIDC Authorization Code + PKCE; xác minh issuer/audience/expiry. Chính sách session theo realm cấu hình, không hardcode trong Booking.

Provider SMS, merchant VNPAY và phiên bản Keycloak SPI chưa được cấp/pin trong nguồn. Schema Redis và hợp đồng adapter ở đây là thiết kế triển khai; cần kiểm chứng với phiên bản thực tế, không tuyên bố đã tích hợp. Không truyền OTP qua Kafka hoặc persist trong audit.

10. State model và kiểm soát đồng thời

Đối tượng	Chuyển trạng thái hợp lệ	Điều kiện
Concert	DRAFT -> PUBLISHED -> HIDDEN -> PUBLISHED	Publish validate dữ liệu; ẩn chỉ chặn giữ mới.
Order	PENDING_PAYMENT -> CONFIRMED / CANCELLED / EXPIRED	Terminal không chuyển ngược. Tới hạn ưu tiên EXPIRED khi xử lý hủy.
Inventory	AVAILABLE -> HELD -> SOLD; HELD -> AVAILABLE; HELD quá hạn -> HELD mới	Đúng hold/order; SOLD terminal. Cấp lại không sửa lịch sử order_item.
Payment attempt	PROCESSING -> VERIFYING/SUCCEEDED/FAILED; VERIFYING -> SUCCEEDED/FAILED	Timeout => VERIFYING. SUCCEEDED không bị callback cũ ghi lùi. FAILED chỉ khi kết quả đã xác minh; bằng chứng provider sửa kết luận phải đi qua đối chiếu bất thường, không qua callback ghi đè trực tiếp.
Refund	PENDING -> SUCCEEDED/NEEDS_SUPPORT; NEEDS_SUPPORT -> PENDING/SUCCEEDED	Không đánh SUCCEEDED chỉ vì đã gửi/accepted.
Ticket	ISSUED -> USED	Một lần, không reset. Chưa phát hành được biểu diễn bằng chưa có row.
Email	PENDING -> SENDING -> SENT/RETRY_REQUIRED/NEEDS_SUPPORT	Hết lease SENDING chưa rõ => ghi UNKNOWN attempt, retry có kiểm soát.
Workflow intent	PENDING -> IN_FLIGHT -> DELIVERED/ATTENTION	Claim hết hạn được retry; vẫn dùng business key cũ.

10.1. Thứ tự khóa Booking

Baseline ưu tiên dễ chứng minh đúng: các thao tác đổi tồn kho serialize theo performance. Nếu có Idempotency-Key, khóa advisory transaction theo key trước mọi row lock (mục 9), rồi đọc lại kết quả cũ. Khóa dữ liệu theo thứ tự: concert (FOR SHARE với checkout, FOR UPDATE khi đổi nội dung/cấu hình) -> performance (FOR UPDATE) -> booking_order nếu đã tồn tại -> performance_seat theo UUID tăng dần -> seat_inventory theo cùng thứ tự -> ticket khi cần. Cấu hình/đóng bán tuân cùng thứ tự; không gọi HTTP/Kafka trong transaction. Serialize mỗi suất là trade-off throughput có chủ đích cho bản local; đo trước khi tối ưu sang khóa ghế tinh hơn.

Đọc trước order để biết performance/concert được phép, sau đó lấy khóa đúng thứ tự và đọc lại trạng thái. Không khóa đơn cũ của ghế hết hạn trong transaction cấp giữ mới: chỉ thay current owner của inventory, để expiry worker xử lý đơn cũ. Nhờ đó tránh chu trình old-order -> seat -> new-order. Optimistic version áp dụng cho sửa cấu hình; 409 nếu If-Match cũ.

Sau khi lấy đủ khóa, lấy clock_timestamp() một lần làm decision_time. Chỉ xác nhận nếu decision_time < expires_at. Điểm quyết định là câu ghi trạng thái có điều kiện sau khóa; transaction ngắn và commit ngay. Không dùng now()/transaction start time cho kiểm hạn sau chờ khóa. Không có cam kết timestamp commit mạng bằng đúng decision_time.

10.2. Khóa Payment và check-in

Payment: payment_order -> payment_attempt -> refund. IPN/query/submit cùng thứ tự. Claim provider I/O bằng lease token, commit trước gọi; cập nhật kết quả dưới khóa, đối chiếu lại trạng thái. Lease hết không là quyền gửi lại refund chưa rõ. Một khoản thu được chọn tại successful_attempt_id; các bằng chứng mâu thuẫn giữ trong observation và chuyển đối chiếu.

Check-in không tranh performance FOR UPDATE để tránh serialize mọi cổng. Khóa assignment active và performance_gate FOR SHARE, khóa performance FOR SHARE để cố định cửa sổ, rồi UPDATE ticket có điều kiện; thay quyền/cửa sổ lấy khóa tương ứng FOR UPDATE. Thứ tự toàn bộ use case check-in: performance -> assignment -> gate -> ticket. Quản trị assignment/gate cũng khóa performance trước. Thu hồi quyền có hiệu lực tại điểm commit; request đã giành shared lock trước có thể hoàn tất.

10.3. Ràng buộc SQL trọng yếu (minh họa)

```
CREATE UNIQUE INDEX ux_payment_one_open
ON payment_attempt(order_id)
WHERE status IN ('PROCESSING', 'VERIFYING');

-- Các unique khác được tạo trong migration tương ứng.
CREATE UNIQUE INDEX ux_ticket_seat
ON ticketing.ticket(performance_seat_id);

UPDATE ticketing.ticket
SET status='USED', used_at=:db_time,
    updated_at=:db_time, version=version+1
WHERE id=:ticket_id AND status='ISSUED'
RETURNING id;
-- Ch INSERT check_in n u UPDATE tr 1 row;
-- INSERT + audit + outbox trong cùng transaction.
```

Đây là đoạn thể hiện constraint/cách ghi, không phải migration chạy độc lập. Các FK vòng seat_inventory -> booking_order và payment_order -> payment_attempt được ADD CONSTRAINT sau khi tạo cả hai bảng. Không dùng CHECK có subquery để giả vờ bảo vệ tổng tiền hoặc trạng thái liên bảng.

11. Thiết kế theo tính năng

Mỗi tính năng dưới đây gồm trigger/API, tiền điều kiện, transaction, xử lý lặp và ngoại lệ. Path là contract đề xuất của LLD; chưa phải OpenAPI đã triển khai. Prefix bên ngoài /api/v1; API nội bộ /internal/v1, bắt buộc service identity.

F01. Tài khoản, quyền và phân công

Truy vết: FR-01/02; PF-01; HLD 10

Mục	Thiết kế
Trigger / API	Đăng ký/login/reset qua Keycloak. PUT /admin/organizers/{id}/members/{subject}; PUT /admin/performances/{id}/assignments/{subject}.
Tiền điều kiện	Admin có role quản trị quyền; subject tồn tại; organizer/performance tồn tại.

1. Keycloak chuẩn hóa phone E.164, kiểm email/username duy nhất, chạy required action email + OTP SMS thật. Chưa xác minh đủ không checkout.

2. Booking kiểm token, validate subject qua Identity trước transaction. Ghi membership/assignment hoặc is_active=false cùng audit. Quyền đối tượng được đọc trực tiếp cho thao tác đặc quyền.

3. UI buyer chỉ nhận dữ liệu mình. Organizer/admin gọi API hỗ trợ có DTO riêng. Nếu không xác minh được scope thì từ chối, không mặc định global admin.

Bất biến / retry: Không lưu password ở Booking. Không dùng buyerId/role client gửi. Không cho admin sửa phone_verified qua API hỗ trợ.

Lỗi: 401 phiên lỗi; 403 thiếu role; 404 đối tượng ngoài scope/không tồn tại; 409 version conflict; 503 Identity/authorization chưa xác minh được.

F02. Tạo concert, ghế, giá và công bố

Truy vết: FR-03/04; PF-02; HLD 6/11

Mục	Thiết kế
Trigger / API	POST /organizer/concerts; PUT /organizer/concerts/{id}; POST .../{id}/publish hoặc /hide; PUT /organizer/performances/{id}/configuration; PATCH .../seats/prices; POST .../close-sales.
Tiền điều kiện	ORGANIZER + MANAGER đúng organizer; If-Match version cho sửa; không đổi lịch/venue suất đã công bố.

1. Draft tạo concert, performance, section, seat và inventory AVAILABLE. Tạo seat/inventory cùng transaction, không để thiếu row quyền.

2. Sửa cấu hình khóa theo mục 10; kiểm lại từng ghế sau lock. Thay giá không đổi order_item. Disable/đổi định danh ghế bị từ chối nếu giữ còn hạn hoặc SOLD.

3. Publish validate title/artists/terms, venue, thời gian, tối thiểu một suất có ghế enabled giá dương và cổng/check-in window. sales_enabled chỉ bật khi cấu hình hợp lệ.

4. Cùng transaction: tăng concert.search_version, ghi CatalogSnapshotChanged và audit; nếu thay mẫu số tồn kho thì tăng inventory_version, ghi InventoryChanged. Hide/close chỉ chặn hold mới; hold cũ vẫn được xác nhận trước hạn.

Bất biến / retry: Một khoảng bán/suất; giá nguyên VND >0; sale window [open,close). Quản trị không sửa snapshot giao dịch.

Lỗi: 422 cấu hình/giá/giờ sai; 409 ghế đang giữ/đã bán hoặc version cũ; 404 ngoài scope.

F03. Tìm kiếm, chi tiết và sơ đồ ghế

Truy vết: FR-05/06; PF-03; HLD 11

Mục	Thiết kế
Trigger / API	GET /concerts/search?q=&venueId=&from=&to=&cursor=; GET /concerts; GET /concerts/{id}; GET /performances/{id}/seats.
Tiền điều kiện	Public, chỉ concert PUBLISHED; khoảng ngày hợp lệ. Không có minPrice/maxPrice.

1. Search query nested theo cùng suất, sort ngày diễn khớp gần nhất rồi ID; lấy candidate IDs.

2. Search batch-check Booking và trả DTO public đã được nguồn xác minh. Filter nội dung ẩn; trả cursor mới theo candidate đã scan, giới hạn scan bù.

3. Seat map đọc Booking/cache nhưng kiểm concert còn public. HELD hết hạn được hiển thị AVAILABLE về mặt khả dụng, không lộ current_order_id/hold_id/buyer. Response có serverTime, readAt.

4. Search unavailable: client chuyển /concerts để duyệt cơ bản; detail và hold không phụ thuộc Elasticsearch.

Bất biến / retry: Trạng thái màn hình chỉ là snapshot; giữ luôn kiểm lại DB. Nội dung đã gửi trước hide không thể thu hồi.

Lỗi: 422 filter lỗi; 404 không public; 503 Search tạm lỗi kèm fallback. Không trả cache khi không kiểm chứng được public scope.

F04. Checkout và giữ 1-4 ghế

Truy vết: FR-07/08/19; PF-03; HLD 6

Mục	Thiết kế
Trigger / API	POST /orders, header Idempotency-Key; body {performanceId,seatIds[]}. Response 201 {orderId,orderCode,status,items,totalVnd,currency,expiresAt,serverTime}.
Tiền điều kiện	BUYER, email_verified + phone_verified; 1-4 seat ID khác nhau; còn lịch bán; rate limit hợp lệ.

1. Canonicalize seatIds theo thứ tự; hash payload gồm performanceId. Kiểm idempotency theo subject/operation/key. Cùng key/hash trả resource cũ trước rate-limit tạo mới.

2. Redis atomic rate-limit theo subject + IP, không là hạn mức tổng vé. Vào transaction: lấy advisory transaction lock theo hash64(subject,operation,key), kiểm lại idempotency để trả kết quả cũ nếu có; sau đó khóa concert/performance, seat config và inventory theo thứ tự.

3. Lấy DB decision_time sau khóa, kiểm PUBLISHED/sales_enabled/window và mọi ghế enabled/cùng suất/AVAILABLE hoặc HELD đã hết hạn. Bất kỳ ghế lỗi rollback hết.

4. Sinh orderId/holdId; expires_at=decision_time+5 phút. Insert booking_order và đủ order_item snapshot. Update toàn bộ seat_inventory HELD/current owner; kiểm số row bằng seat_count.

5. Ghi idempotency response, workflow_intent START_ORDER, OrderCreated và InventoryChanged outbox; tăng inventory_version; commit. Không gọi Temporal/provider trong transaction. Response bị mất được retry cùng key.

Bất biến / retry: Một transaction giữ đủ hoặc không giữ; phí 0; không reset expiry. Không giải phóng hoặc cập nhật order cũ khi cấp lại ghế quá hạn.

Lỗi: 409 SEAT_UNAVAILABLE/IDEMPOTENCY_CONFLICT/SALES_CLOSED; 422 INVALID_SEAT_SET; 429 RATE_LIMITED; 503 BUSY hoặc RATE_LIMIT_UNAVAILABLE. Timeout kết quả chưa rõ => retry cùng key/tra đơn.

F05. Bắt đầu thanh toán VNPAY sandbox

Truy vết: FR-09; PF-04; HLD 8

Mục	Thiết kế
Trigger / API	POST /orders/{id}/payment-attempts; Booking -> POST /internal/v1/payment-attempts. Body nội bộ {orderId,amountVnd,currency,scope,holdExpiresAt,operationKey}.
Tiền điều kiện	Chủ đơn, PENDING_PAYMENT và còn quyền hold; tiền lấy từ Booking snapshot; không nhận giá client.

1. Booking xác minh quyền và trạng thái hiện tại, tạo operation key ổn định. Gọi Payment ngoài transaction khóa ghế. Retry HTTP dùng lại key.

2. Payment tạo/lấy payment_order rồi khóa row. Cùng key/hash trả attempt cũ. Nếu có PROCESSING/VERIFYING trả attempt đó; nếu đã có thu thành công không mở lần mới. Chỉ sau FAILED rõ và còn hạn mới tăng attempt_no.

3. Commit attempt PROCESSING/provider_ref trước trả URL. Adapter dựng URL đã ký từ snapshot, lưu URL được bảo vệ để retry trả cùng ref. Không dựng provider_ref khác vì mất response.

4. UI mở URL provider; QR thanh toán do provider cấp. Return URL chỉ điều hướng đọc trạng thái đơn, không thay đổi tiền/đơn. Nếu hủy/hết hạn xảy ra sau kiểm trước HTTP, tiền thu đến sau sẽ bù trừ.

Bất biến / retry: Một open attempt/đơn bằng partial unique + khóa payment_order; không kéo dài hold. Tham số amount provider được adapter chuyển từ nguyên VND theo hợp đồng HLD.

Lỗi: 409 ORDER_NOT_PAYABLE/ATTEMPT_UNRESOLVED; 503 provider chưa truy cập được. URL timeout không cho tạo lần thu mới nếu kết quả cũ chưa rõ.

F06. IPN, query và chốt đơn

Truy vết: FR-09/20; PF-04; HLD 6-8

Mục	Thiết kế
Trigger / API	GET /payments/vnpay/ipn (method cuối theo provider contract); Payment reconciliation job; Kafka PaymentStateChanged -> Booking -> Temporal activity.
Tiền điều kiện	Xác minh chữ ký, merchant, reference, số tiền/VND, trạng thái nguồn. Không tin Return URL.

1. Payment lưu observation đã sanitize; dưới lock payment_order/attempt ghi trạng thái hợp lệ + paid_at/verified_at + outbox/audit. Gửi acknowledgement provider sau commit. Callback sai/mâu thuẫn không ghi đè kết quả.

2. Booking consumer commit inbox + payment_view + workflow_intent signal. Signal có thể đến trước START: dispatcher start-or-signal theo workflow ID; activity dedup theo event và trạng thái DB.

3. Confirm activity khóa đúng thứ tự, lấy decision_time sau khóa. Nếu CONFIRMED cùng attempt: no-op. Nếu pending, còn hạn, đủ owner/hold, amount khớp: cùng transaction order CONFIRMED, toàn bộ ghế SOLD, confirmed_attempt_id và outbox phát hành vé.

4. Nếu tới hạn: EXPIRED; nếu mất quyền giữ trước hạn: CANCELLED/INVENTORY_INCONSISTENCY + needs_attention. Release phần còn đúng hold; tạo nghĩa vụ refund qua intent/activity, không phục hồi đơn.

5. Nếu đơn đã CONFIRMED bằng attempt khác: khoản thu mới là EXCESS_CHARGE, yêu cầu refund khoản mới và audit; không refund khoản đã dùng để xác nhận. Observation xung đột được query nguồn trước công nhận tiền mới.

Bất biến / retry: Thời điểm callback không thay decision_time. Không confirm một phần. Sự kiện trạng thái cũ không làm lùi projection; tiền thật đã xác minh vẫn được lưu dù bất thường.

Lỗi: IPN invalid => acknowledgement lỗi theo adapter, không HTTP 200 thành công nghiệp vụ tùy tiện. Amount mismatch => QUARANTINED/needs_attention. DB/Temporal lỗi giữ intent để retry.

F07. Hủy và hết hạn

Truy vết: FR-10; PF-05; HLD 6/7

Mục	Thiết kế
Trigger / API	POST /orders/{id}/cancel; Temporal timer; Booking expiry reconciliation quét pending đã quá hạn.
Tiền điều kiện	Chủ đơn cho cancel; hệ thống cho expire. Đơn CONFIRMED không cho hủy.

1. Lấy khóa concert/performance/order/seats theo chuẩn; đọc giờ DB sau khóa. Pending trước hạn + cancel => CANCELLED; pending tới/sau hạn => EXPIRED.

2. Update chỉ seat_inventory HELD có current_order_id và hold_id trùng. Không chạm SOLD hoặc hold mới. Ghi outbox InventoryChanged/OrderClosed cùng audit.

3. Payment đang chưa rõ vẫn query độc lập. Khi có tiền SUCCEEDED gửi/tái thử refund theo attempt; nếu không có thu thì không tạo refund. Timer lặp hoặc cleanup chậm không thay hạn.

Bất biến / retry: Terminal không phục hồi. Race cancel/confirm chọn một nhánh dưới cùng lock. Workflow cleanup cũ không giải phóng ghế của người khác.

Lỗi: 409 ORDER_ALREADY_CONFIRMED; cancel lặp trả 200 trạng thái đã kết thúc. 503 khi chưa ghi được; client không tự kết luận đã hủy.

F08. Hoàn tiền bù trừ

Truy vết: FR-11; PF-06; HLD 8

Mục	Thiết kế
Trigger / API	Booking activity -> POST /internal/v1/refunds; admin retry qua support operation. Body {attemptId,reason,operationId}; amount do Payment đọc.
Tiền điều kiện	Khoản thu SUCCEEDED đã xác minh; Booking xác nhận đơn terminal hoặc EXCESS_CHARGE có bằng chứng; không hoàn từng vé.

1. Khóa payment_order/attempt; tạo hoặc lấy refund theo UK attempt. Lưu amount toàn bộ, reference cố định, provider_state=NOT_SENT, PENDING cùng audit/outbox.

2. Worker claim/refund; commit provider_call STARTED và trạng thái gửi trước I/O. Gọi provider ngoài transaction. Nếu response chỉ ACCEPTED: giữ PENDING, lên lịch query.

3. Timeout hoặc crash giữa call và save: đặt UNKNOWN hoặc suy ra từ STARTED chưa xong; chỉ đối chiếu reference cũ. Không gửi request mới khi chưa chứng minh an toàn theo provider contract.

4. Kết quả hoàn đã xác minh => SUCCEEDED/refunded_at, outbox RefundStateChanged và email kết quả. Hết khả năng tự xử lý => NEEDS_SUPPORT, lưu lý do; admin retry vẫn dùng refund cũ.

Bất biến / retry: UK một nghĩa vụ/attempt; SUCCEEDED terminal; refund không làm Payment SUCCEEDED thành FAILED và không cấp vé. Query/refund chưa được merchant hỗ trợ => phần tích hợp chưa hoàn tất, không giả lập thành công.

Lỗi: 409 REFUND_NOT_ALLOWED/AMOUNT_MISMATCH; 202 đã tiếp nhận không đồng nghĩa đã hoàn; timeout => PENDING/NEEDS_SUPPORT.

F09. Phát hành, xem vé và QR

Truy vết: FR-12/13; PF-07; HLD 9

Mục	Thiết kế
Trigger / API	IssueTickets activity; GET /me/orders; GET /me/orders/{id}; GET /me/tickets/{id}/qr. Nội bộ GET /internal/v1/notification-grants/{id}/ticket-content.
Tiền điều kiện	Order CONFIRMED, seat SOLD đúng order; owner cho đọc QR; Notification service cho endpoint grant.

1. Activity khóa order (không sửa inventory), xác minh lines đã bán; tạo mỗi ticket nếu chưa tồn tại. Sinh token random 32 byte một lần, lưu digest + ciphertext/nonce/keyId. UK line và seat bảo vệ retry.

2. Sau mỗi transaction nhỏ có thể đã phát một phần; retry chỉ bổ sung thiếu. Khi count(ticket)=seat_count, dưới order lock đặt ticket_set_issued_at một lần, tạo notification_grant và TicketSetIssued outbox cùng transaction.

3. Owner API join booking_order.buyer_subject, decrypt token, render PNG trong memory. Cache-Control: no-store; không URL bucket public, không token trong query string hoặc log.

4. Notification content API kiểm service identity + grant/order/owner/recipient, trả dữ liệu đủ gửi đúng email. Admin/organizer chỉ có metadata DTO; không có đường xem QR qua lịch sử email hoặc support response.

Bất biến / retry: QR cũ dùng lại khi resend/retry. Order vẫn CONFIRMED khi vé đang phát. Không có bảng ticket_file hoặc trạng thái PDF.

Lỗi: 404 vé không thuộc chủ/không tồn tại; 409 TICKETS_NOT_READY; 503 decrypt/render tạm lỗi, giữ nguyên vé/đơn.

F10. Check-in online

Truy vết: FR-14; PF-08; HLD 9

Mục	Thiết kế
Trigger / API	POST /check-ins body {performanceId,gateId,qrToken}; token trong body, request logging redact. Response 200 CHECKED_IN hoặc 409 ALREADY_USED.
Tiền điều kiện	CHECKIN_STAFF + assignment active; cổng enabled đúng suất; cửa sổ [checkin_open_at,checkin_close_at).

1. Kiểm định dạng/giới hạn token, SHA-256 tra ticket. Lấy shared lock performance/assignment/gate; kiểm lại quyền/cửa sổ bằng DB time.

2. UPDATE ticket ISSUED -> USED RETURNING. Nếu 1 row: INSERT check_in PK ticket_id + audit + TicketCheckedIn outbox cùng transaction; trả thời gian/cổng.

3. Nếu không update vì USED: đọc check_in đã commit, trả ALREADY_USED với lần dùng trước; không tạo row check_in mới. Không lộ thông tin vé sai suất trước khi kiểm scope.

4. Mất response sau commit: lần quét sau trả lần dùng cũ. Không có endpoint reset USED, check-in offline hoặc override admin.

Bất biến / retry: Một record check_in/ticket. Transaction fail rollback cả status và record. QR không được ghi trong audit.

Lỗi: 403 không được giao; 422 INVALID_QR/WRONG_PERFORMANCE/INVALID_GATE; 409 OUTSIDE_CHECKIN_WINDOW/ALREADY_USED; 503 UNVERIFIABLE khi mạng/DB lỗi, không báo vé giả.

F11. Email và hỗ trợ giao dịch

Truy vết: FR-15/16/18; PF-07/09; HLD 9/13

Mục	Thiết kế
Trigger / API	Kafka TicketSetIssued/RefundStateChanged; GET /support/orders?...; POST /support/operations; GET /support/operations/{id}.
Tiền điều kiện	Email refund chỉ khi SUCCEEDED; hỗ trợ cần ADMIN + organizer scope, reason bắt buộc. Organizer chỉ đọc scope của mình.

1. Notification ghi inbox + notification_request trong cùng transaction, dedup business_key. Worker claim, fetch content bằng grant cho QR; fetch recipient nguồn cho refund. Render/gửi ngoài transaction.

2. Ghi attempt trước SMTP; sau accept mới SENT. Lỗi rõ retry; timeout/crash => UNKNOWN và retry có kiểm soát, chấp nhận nguy cơ email lặp. Không đổi quyền vé/đơn.

3. Support lookup mã order/ticket/payment/refund hoặc buyer subject/email chính xác. Nếu bắt đầu từ payment reference: Payment resolve order qua API nội bộ, Booking kiểm scope trước trả DTO tổng hợp.

4. Admin tạo support_operation + audit + intent/outbox cùng transaction. Retry các hành động trong whitelist; với resend, tạo grant/key mới theo operationId nhưng QR và recipient gốc không đổi. Chỉ trả metadata gửi và lỗi đã che thông tin.

Bất biến / retry: Không ép confirm/refund success; không sửa recipient tùy ý. Replay event khác resend chủ đích. Audit từ chối được ghi sau rollback bằng transaction riêng.

Lỗi: 404 ngoài scope; 409 ACTION_NOT_ALLOWED; 202 ACCEPTED kèm operationId; trạng thái thành phần không đọc được trả unavailable, không giả định FAILED.

F12. Báo cáo, replay và thực nghiệm

Truy vết: FR-17/20; PF-10/11; HLD 12/15/17

Mục	Thiết kế
Trigger / API	GET /reports/summary?performanceId=&fromDate=&toDate=; GET /reports/details; scheduled Spark batch; run replay theo manifest.
Tiền điều kiện	Scope organizer được xác minh tại Booking; input period hợp lệ; active run đã hoàn tất.

1. Ingest/export/batch theo mục 8; tính tiền/vé/check-in theo business time, occupancy cùng snapshot. Replay ghi run mới, không cộng dồn run cũ.

2. API đọc activeRun một lần, query tất cả chỉ số bằng runId đó. Trả runId, watermark theo nguồn, business cutoff và trạng thái stale. Không tự dùng giờ query làm updated-to.

3. Drill-down theo business references trong scope; Report chỉ giải thích chênh lệch, không ghi vào Booking/Payment.

4. Thực nghiệm ghi môi trường/commit/cấu hình/dataset/workload/fault point và kết quả p50/p95/p99, throughput, error, recovery; đối chiếu nguồn với report theo cùng watermark. Không tự cam kết TPS hoặc SLA chưa đo.

Bất biến / retry: Duplicate/replay không tăng tổng; watermark chưa đủ không được trình bày như dữ liệu hoàn chỉnh. Lỗi Report không chặn checkout.

Lỗi: 422 period lỗi; 403/404 ngoài scope; 503 chưa có run; run cũ có stale=true khi batch lỗi.

12. Hợp đồng API và sự kiện

12.1. Quy ước REST

API ngoài trả JSON camelCase, ID chuỗi UUID, tiền integer VND, timestamp ISO-8601 UTC. Authentication Bearer token trừ catalog public và IPN. Header X-Correlation-Id là trace, không thay Idempotency-Key; nếu client không gửi correlation, server sinh UUID. Cursor opaque, default pageSize=20, max=100 là baseline LLD có cấu hình.

Nhóm	Request / response tối thiểu
Create order	Request: performanceId, seatIds[]. Server bỏ qua/không nhận buyerId, price. Response: orderId, orderCode, status, items[{seatId,section,row,number,unitPriceVnd}], feeVnd=0, totalVnd, currency, heldAt, expiresAt, serverTime.
Order detail	orderStatus, payment{status,attemptId,lastUpdatedAt,available}, refund{status,refundId,available}, tickets{issuedCount,expectedCount}, email{status,available}. Chưa có payment = NOT_STARTED; chưa cần refund = NOT_APPLICABLE, không persist hai giá trị này như giao dịch.
Payment start	Response: attemptId, paymentStatus, paymentUrl (chỉ owner), expiresAt. Cùng operation trả attempt cũ; không echo URL vào audit/idempotency JSON.
Check-in	Request: performanceId, gateId, qrToken. Response: result, ticketCode, seatLabel, checkedInAt, gateId; không echo token. ALREADY_USED có usedAt/priorGate.
Support	operationId, action, state, targetReference, sourceStatus, nextAction. SourceStatus unavailable được biểu diễn rõ; reason là input bắt buộc.
Report	runId, sourceWatermarks[], businessCutoff, stale, period, currency=VND, metrics, occupancyAsOf.

Error envelope: {code,message,correlationId,retryable,details}. Không stack trace/secret. 400 JSON sai; 401 chưa xác thực; 403 thiếu role; 404 resource không thuộc scope; 409 state/idempotency/version conflict; 422 dữ liệu hợp lệ về JSON nhưng sai nghiệp vụ; 429 rate limit có Retry-After; 503 nguồn phụ thuộc/lock timeout. Response 202 chỉ là tiếp nhận tác vụ async. IPN có response riêng theo provider contract, không dùng error

envelope người mua.

12.2. API nội bộ và quyền gọi

API	Caller	Cơ chế
POST /internal/v1/payment-attempts	Booking	Snapshot tiền/hold do Booking cung cấp; Payment kiểm hash và state dưới khóa.
GET /internal/v1/payment-orders/{id}	Booking	Trạng thái các attempt/refund; không credential/provider secret.
POST /internal/v1/refunds	Booking	attemptId/reason/operationId; Payment tự lấy amount.
POST /internal/v1/payments/{id}/verify	Booking support dispatcher	Tiếp nhận query lại giao dịch cũ; operationId dedup.
POST /internal/v1/public-concerts/resolve	Search	Batch IDs; Booking trả DTO public của ID còn được phép.
GET /internal/v1/catalog/snapshots	Search reindex worker	Snapshot paginated + search_version; bao gồm tombstones cho worker có quyền.
POST /internal/v1/authorization/resolve	Payment/Report/Notification	Subject, action, object/scope; caller identity bắt buộc. Không endpoint public tự xin scope.
GET /internal/v1/notification-grants/{id}/ticket-content	Notification	Grant + owner + recipient binding; chỉ purpose email có QR.
GET /internal/v1/orders/{id}/notification-recipient	Notification	Chỉ notification type/ref hợp lệ; trả snapshot email đã xác minh.
GET /internal/v1/inventory/snapshots	Report replay worker	Baseline performance version + từng seat state/enabled tại nhất quán DB.

12.3. Event catalog

Envelope v1: eventId UUID; eventType; schemaVersion integer; producer; aggregateType; aggregateId; aggregateVersion; occurredAt UTC; correlationId; causationId; payload. Trace truyền qua activity/consumer, không chỉ ThreadLocal. Consumer từ chối schemaVersion chưa hỗ trợ vào quarantine có cảnh báo; không bỏ event rồi commit offset âm thầm.

Topic / partition key	Event / payload nghiệp vụ	Consumer
booking.catalog.v1 / concertId	CatalogSnapshotChanged: full snapshot concert + performances, isPublic, searchVersion; giữ tombstone khi HIDDEN.	Search, Report
booking.inventory.v1 / performanceId	SeatMapInitialized hoặc InventoryChanged: inventoryVersion, effectiveAt, seatChanges[{seatId,enabled,statusAfter}], organizerId, concertId.	Report
booking.order.v1 / orderId	OrderCreated/OrderConfirmed/OrderClosed: orderId, buyerSubject chỉ khi consumer cần, scope, status, amountVnd, currency, expiresAt, confirmedAttemptId?, reason?.	Report; Ticketing do workflow activity điều phối
payment.payment.v1 / orderId	PaymentStateChanged: attemptId, orderId, amountVnd, currency, status, paidAt?, verifiedAt?, scope, attemptVersion. Full snapshot.	Booking, Report
payment.refund.v1 / orderId	RefundStateChanged: refundId, attemptId, orderId, amountVnd, currency, status, refundedAt?, scope, refundVersion.	Booking, Report, Notification (chỉ SUCCEEDED)

Topic / partition key	Event / payload nghiệp vụ	Consumer
booking.ticket.v1 / orderId	TicketIssued: ticketId, orderId, seatId, performanceId, organizerId, issuedAt. TicketSetIssued: orderId, count, notificationKey, grantId. TicketCheckedIn: ticketId, orderId, performanceId, organizerId, gateId, checkedInAt.	Report; Notification chỉ TicketSetIssued
booking.support.v1 / orderId	EmailResendRequested: operationId, orderId, originalRequestId?, notificationKey, grantId?, type.	Notification

Không phát email recipient/QR/password/OTP/token qua topic chung. Notification có thể chưa lấy được recipient khi nhận event: commit inbox PENDING trước, worker fetch nguồn rồi transaction tạo request + chuyển APPLIED. TicketSetIssued chỉ phát một lần theo order marker; event bị relay lặp được inbox/business_key chặn. Một event có nhiều consumer group; ACL topic/service được giới hạn theo nhiệm vụ.

Đơn confirmed chưa phát đủ vé được Temporal activity retry; không vừa dùng consumer Ticketing độc lập vừa workflow chạy hai hệ điều phối khác nhau. TicketIssued có eventId riêng cho từng vé; stream_seq phân biệt event cùng aggregate version nếu cần. Kafka key orderId không thay aggregateVersion của ticket/refund; consumer so version trong đúng aggregate, không so version giữa hai loại.

12.4. Workflow và phục hồi khỏi khoảng trống giao dịch

Workflow / activity	Trách nhiệm và kết quả
OrderWorkflow: order:{orderId}	Start từ workflow_intent; timer đến expires_at gốc; signal payment/refund/support bằng eventId. Giữ liveness khi tiền chưa rõ.
ConfirmOrder / ExpireOrder / CancelOrder	Gọi use case Booking transaction; activity retry trả trạng thái hiện tại, không tái phát tác động. Cancel API commit kết quả rồi workflow đọc lại.
RequestRefund	HTTP Payment dùng khóa refund theo attempt; retry lấy nghĩa vụ cũ. Payment tự query provider; workflow không có polling provider thứ hai.
IssueTickets	Bổ sung vé thiếu; marker đủ vé + grant + outbox atomic. Thất bại không refund đơn đã xác nhận.
Workflow kết thúc / sự kiện muộn	Chỉ đóng bình thường khi pending intents đã được xử lý và các khoản tiền đã rõ. Event phát sinh rất muộn sau đóng vẫn được giữ trong inbox/intent; dispatcher khởi động recovery execution theo cùng business order (ID reuse policy đã cấu hình) hoặc gửi vào execution đang chạy. Activity luôn đọc DB, không phục hồi quyền hold.
History dài	Continue-as-new giữ business order ID, expiry và reference cần thiết; không reset thời hạn giữ. Signal lặp dựa eventId + idempotent state mutation; không giữ tập dedup vô hạn trong workflow memory.

Không coi Temporal start/signal hoặc SMTP/provider call là một phần transaction PostgreSQL. Delivery đều có thể lặp. Khi workflow chưa chạy, expiry reconciliation vẫn kết thúc order quá hạn; tín hiệu payment đến sau vẫn phải đi tới refund hoặc hồ sơ cần hỗ trợ, không bị bỏ vì workflow cũ đã đóng.

13. Migration, vận hành và các điểm cần khóa khi triển khai

13.1. Liquibase theo owner

Change set	Phạm vi / thứ tự
Booking 001	Tạo schema platform/catalog/inventory/ordering/ticketing; organizer/member; venue/gate; concert/performance; gate/assignment; section/seat.
Booking 002	booking_order -> order_item -> seat_inventory -> ticket -> check_in; composite unique phải có trước FK; sau đó payment_view, workflow_intent, grant, support_operation.
Booking 003	Idempotency/outbox/inbox/audit; index/constraint; seed venue/gate và dataset demo riêng profile local.
Payment 001-003	payment_order (chưa successful_attempt FK) -> payment_attempt -> refund -> ADD FK successful_attempt -> provider tables -> common tables + shedlock.
Notification 001-002	notification_request -> attempt -> inbox/audit và chỉ mục; template version quản lý cùng release.
Search / Report	Index template/alias và Mongo validators/index bằng script versioned riêng; Liquibase không quản lý ES/Mongo/MinIO.
Identity / Temporal	Migration nội bộ theo image/version được pin. Không sửa schema sản phẩm qua application changelog.

Không sửa change set đã chạy; thay đổi bằng change set mới. Dữ liệu demo và fault switches tách khỏi migration cấu trúc. Khi thêm NOT NULL vào bảng có dữ liệu: add nullable -> backfill có kiểm soát -> validate -> enforce. Backup trước migration không đảo ngược; không dùng drop schema làm rollback dữ liệu nghiệp vụ.

13.2. Cấu hình baseline LLD

Cấu hình	Giá trị / cách xử lý
hold.duration / maxSeats	5 phút / 4, là quy tắc nghiệp vụ đã chốt.
booking.lock_timeout	Khởi điểm 2 giây; transaction deadline 5 giây. Không chờ khóa vô hạn; 503 BUSY và retry cùng key. Cần hiệu chỉnh theo tải thực.
DB deadlock retry	Tối đa 2 lần ở use case idempotent, jitter ngắn; không retry external side effect trong transaction retry.
HTTP nội bộ	Connect 2s / read 5s làm baseline local; response chưa rõ dùng lookup. Không nối timeout dài hơn hold rồi vẫn confirm sau hạn.
Worker backoff	Khởi điểm 1s, exponential có jitter, cap 60s; ngưỡng tự động 10 lỗi liên tiếp => needs_attention. Công việc chưa rõ vẫn tồn tại, không bị đánh FAILED để dọn.
Rate limit	Key subject + IP; ngưỡng bắt buộc config khi chạy. Đo rồi chốt; không biến limit thành hạn mức vé/tài khoản. Redis unavailable chặn hold mới, không chặn đọc đơn/check-in có nguồn quyền còn hoạt động.
Lease / ShedLock	Claim hết hạn phải có fencing update. Giá trị lease theo timeout I/O và batch; không coi lease là idempotency provider. Lock job cần gia hạn nếu batch vượt giới hạn.
Retention	Chưa tự động xóa dữ liệu nghiệp vụ/audit/idempotency/raw event; lifecycle raw/manifest và OTP TTL tách riêng. Volume local cần backup, không mặc định chống mất máy.

13.3. Phụ thuộc chưa được xác nhận bằng tích hợp

Thiết kế bảng/luồng có thể triển khai ngay. Những mục sau là đầu vào vận hành hoặc hợp đồng adapter cần kiểm chứng, không được tự suy đoán là đã có:

Đầu vào	Ảnh hưởng
Merchant VNPAY sandbox	Credentials, IPN/Return URL HTTPS, phương thức QR, giới hạn amount/ref, mã response và timezone provider. Adapter validate trước publish URL; không pin chiều dài provider contract chỉ theo varchar lưu trữ.
Refund/query contract	Cần xác nhận query refund chưa rõ, request reference và điều kiện resend an toàn. Nếu chưa hỗ trợ thì NEEDS_SUPPORT; không gọi đi gọi lại như provider có idempotency mặc định.
SMS provider thật	Account, sender/template được duyệt, số nhận Việt Nam và budget; không đánh nghiệm thu chỉ bằng mock.
Phiên bản stack	HLD chọn Java 25/Boot 3.5.x; pin patch/image và kiểm SDK/driver/SPI khi khởi tạo. LLD không tái khẳng định tương thích chưa chạy.
Catalog snapshot export	Quy định snapshot boundary + offset khi reindex; full snapshot per concert có version, không dump DB xuyên service.
Nguồn tài liệu	SRS còn các AC cũ về PDF/lọc giá/simulator. Dùng bảng thay đổi HLD khi thực hiện; cập nhật SRS ở lần chỉnh tài liệu riêng.

14. Truy vết và kiểm chứng thiết kế

Yêu cầu	LLD / bảng chính
FR-01/02	F01; organizer_member, performance_assignment; Keycloak/OTP store.
FR-03/04	F02; concert, performance, venue/gate, section/seat; audit.
FR-05/06	F03; concert_v1, seat_inventory; Booking public validation.
FR-07/08/19	F04; booking_order, order_item, seat_inventory, idempotency; rate limit.
FR-09/10	F05-F07; payment_order/attempt, payment_view, workflow_intent, observation.
FR-11	F08; refund, provider_call/observation, audit.
FR-12/13	F09; ticket, notification_grant; không có ticket_file theo HLD.
FR-14	F10; ticket, check_in, performance_gate/assignment.
FR-15/16/18	F11; notification_request/attempt, support_operation, audit.
FR-17/20	F12; raw_event/manifest, report_run/pointer, daily_metric/occupancy/detail; outbox/inbox/intent.

14.1. Kịch bản kiểm chứng transaction và recovery

Đây là các tình huống E2E/load/fault để kiểm chứng thiết kế, không thêm bộ unit test vào phạm vi tài liệu.

Điểm gây lỗi / cạnh tranh	Kết quả phải quan sát được
Hai buyer giữ một ghế; 4 ghế có 1 bận	Một buyer thắng; tập 4 ghế bị lỗi rollback cả đơn và giữ.
Mất response checkout; retry cùng key	Một order/hold, snapshot và expires_at cũ; payload khác => conflict.
Chờ row lock qua hạn 5 phút	decision_time sau khóa >= expiry: không confirm; tiền đã thu đi refund.
Giữ mới trên ghế cũ hết hạn rồi chạy timer cũ	Timer cũ không giải phóng current owner mới.

Điểm gây lỗi / cạnh tranh	Kết quả phải quan sát được
Cancel đồng thời PaymentSuccess	Một trạng thái đơn terminal; confirm thắng thì không hủy, cancel thắng thì refund.
IPN trùng/sai amount/mâu thuẫn query	Không thu/cộng lại; sai amount không confirm; evidence giữ để đối chiếu.
Provider nhận refund rồi mất response	Không tạo refund/reference mới; query nghĩa vụ cũ hoặc needs_support.
Crash sau commit order trước Temporal start	Intent còn, dispatcher start lại; expiry gốc không đổi.
Crash sau signal trước mark delivered	Signal lặp không tạo thêm confirm/refund/ticket.
Phát 2/4 vé rồi restart	Cuối cùng đúng 4 vé; TicketSetIssued một tác động nghiệp vụ; QR cũ giữ nguyên.
100 scan đồng thời, mất response scan thắng	Một check_in; quét lại ALREADY_USED cùng thông tin cũ.
SMTP accept rồi crash	Có thể email lặp, không thêm vé và không đổi quyền ghế.
Hide concert khi ES còn bản cũ	Không trả nội dung ẩn sau khi Booking validation thấy HIDDEN.
Ghi object xong trước manifest READY	Recovery cùng key/checksum, không mất raw hoặc tính hai lần.
Batch lỗi giữa output và active pointer	API đọc nguyên run cũ hoặc nguyên run mới, không trộn.
Replay thu ngày D, hoàn D+1	Thu/hoàn về đúng kỳ, không cộng trùng; net ngày hoàn có thể âm.
Admin/organizer truy cập QR hoặc đổi recipient	Từ chối; DTO/log/audit không có QR; resend vẫn dùng recipient nguồn.

14.2. Giới hạn của bản thiết kế

Chưa có bằng chứng chạy migration, tích hợp VNPAY/SMS hoặc số đo tải trong lần bàn giao này. Serialize tồn kho theo performance được chọn để giảm độ phức tạp; chỉ tối ưu sau khi có kết quả đo và vẫn giữ các bất biến. QR trong email là bearer credential nên tài khoản truy cập Mailpit/DB/key phải tách khỏi vai trò admin nghiệp vụ; kiểm soát API không bảo vệ được người có toàn quyền hạ tầng.

Tài liệu hoàn tất ở mức thiết kế chi tiết. Các source BRD/Process Flows/SRS/HLD và file mẫu được giữ nguyên; tên bảng, API và baseline vận hành mới ở đây là đầu vào cho migration và implementation tiếp theo.